

TAGD-02-ai

Filippo Visconti

Contents

1	Gestione delle transazioni	3
1.1	Architettura del sistema transazionale	3
1.2	Definizione di transazione	3
1.3	Le proprietà ACID	4
1.4	Corrispondenza tra proprietà e moduli DBMS	5
2	Gestione dell'affidabilità: Atomicità e Durabilità	5
2.1	Persistenza delle memorie e tipologie di guasto	5
2.2	Il modello “fail-stop”	6
2.3	Il Gestore dell'affidabilità	6
2.4	Strumenti di ridondanza: Log e Dump	6
2.5	Modello di riferimento e operazioni	6
2.6	Il Log: Struttura e regole di scrittura	6
2.7	La procedura di Rollback	7
3	Gestione dell'affidabilità: Procedure di Recovery e Ripristino	7
3.1	L'algoritmo di Recovery	8
3.2	Ottimizzazione tramite Checkpoint	8
3.3	Esempio di ripresa a caldo	9
3.4	Tipologie di guasti e procedure di backup	10
4	Controllo della concorrenza	10
4.1	La perdita di aggiornamento (Lost Update)	10
4.2	La lettura sporca (Dirty Read)	10
4.3	Lecture inconsistenti e aggiornamento fantasma	11
4.4	L'inserimento fantasma (Phantom)	11
4.5	Sintesi delle anomalie di concorrenza	11
5	Teoria e pratica del controllo della concorrenza nelle basi di dati	12
5.1	Architettura del Gestore della Concorrenza	12
5.2	Teoria del controllo di concorrenza: Schedule e Serializzabilità	12
5.3	Conflict-Serializzabilità	13
6	Relazioni tra Conflict-Serializzabilità e View-Serializzabilità	13
6.1	Verifica della Conflict-Serializzabilità tramite Grafi	14
6.2	Esempio di Grafo dei Conflitti	14
6.3	Meccanismi di Locking	15
6.4	Fallimenti e Locking a Due Fasi Stretto (S2PL)	15
6.5	Granularità e Stallo	15
6.6	Livelli di Isolamento e Inserimenti Fantasma	16
7	Controllo di concorrenza basato su timestamp	16
7.1	Meccanismi di controllo e variabili di stato	16
7.2	Confronto tra 2PL e Timestamp	17
7.3	Controllo di concorrenza multiversione (MVCC)	17
7.4	Implementazioni nei DBMS reali: DB2 e PostgreSQL	18
7.5	Analisi delle anomalie e sperimentazione pratica	18

8	Gestione delle transazioni nelle basi di dati distribuite	18
8.1	Architetture distribuite: Partizionamento e Replicazione	19
8.2	Tipologie di transazioni distribuite	19
8.3	Consistenza e Durabilità nei sistemi distribuiti	20
8.4	Controllo della concorrenza globale	20
8.5	Tipologie di guasto nei sistemi distribuiti	20
8.6	Il protocollo di Commit a Due Fasi (2PC)	21
8.7	Prima fase del protocollo: Preparazione	21
8.8	Seconda fase del protocollo: Conferma	22
8.9	Diagramma temporale del Commit a due fasi	22
8.10	Incertezza e procedure di ripristino	22
8.11	Gestione dei guasti e dei messaggi perduti	23
9	Consistenza nelle basi di dati distribuite	23
9.1	Partizionamento e Replicazione	23
9.2	Sistemi distribuiti: il trade-off tra Safety e Liveness	23
9.3	Il Teorema CAP	24
9.4	Il problema del Consenso	24
9.5	L'algoritmo RAFT	25
10	Architettura e gestione delle transazioni in SimpleDB	27
10.1	Struttura delle classi per la gestione transazionale	27
10.2	La gestione dei Lock tramite la LockTable	27
10.3	Flussi operativi: Scansione e Modifica dei dati	28
10.4	Conclusione della transazione e rilascio dei Lock	28
10.5	Strategie alternative di acquisizione dei Lock	29

1 Gestione delle transazioni

Le basi di dati rappresentano una risorsa critica per le organizzazioni che le possiedono e devono pertanto essere conservate con estrema cura, **garantendo l'integrità del dato anche a fronte di malfunzionamenti hardware o software**. Un esempio classico è rappresentato dal trasferimento di fondi tra due conti correnti bancari: se il sistema dovesse subire un guasto esattamente a metà dell'operazione, si rischierebbe di aver sottratto denaro da un conto senza averlo accreditato sull'altro. Per ovviare a tali rischi, è necessario un supporto robusto alla gestione delle **transazioni**. Queste devono essere **atomiche**, ovvero eseguite secondo il principio del “tutto o niente”, **e definitive**, garantendo che una volta conclusa l'operazione, i risultati non vengano persi.

L'affidabilità è un obiettivo particolarmente impegnativo da raggiungere a causa della **frequenza elevata** degli aggiornamenti e della necessità tecnica di gestire correttamente il buffer, ovvero la memoria temporanea che intercorre tra l'applicazione e la memoria secondaria. Inoltre, le basi di dati sono per definizione risorse integrate e condivise tra diverse applicazioni. Questa condivisione comporta due conseguenze fondamentali: la necessità di meccanismi di autorizzazione per gestire attività diverse su dati parzialmente comuni e la necessità di un rigoroso controllo della concorrenza per gestire attività multi-utente simultanee.

Senza un controllo adeguato, gli aggiornamenti su basi di dati condivise potrebbero generare anomalie. Si pensi a due prelevamenti quasi contemporanei sullo stesso conto corrente o a due prenotazioni simultanee per lo stesso posto su un volo. Intuitivamente, una collezione di transazioni è corretta se viene eseguita in modo seriale, cioè una dopo l'altra. Tuttavia, nei sistemi reali, l'efficienza verrebbe compromessa in modo inaccettabile se non fosse permesso il parallelismo; il **controllo della concorrenza** agisce quindi come un **compromesso ragionevole per mantenere l'efficienza senza sacrificare la correttezza**.

1.1 Architettura del sistema transazionale

L'organizzazione modulare di un DBMS per la gestione degli accessi e delle transazioni prevede un'interazione complessa tra diversi componenti. Il flusso principale parte dal Gestore di Interrogazioni e aggiornamenti, che si interfaccia con il Gestore dei metodi d'accesso. Quest'ultimo comunica con il Gestore del buffer, il quale a sua volta interagisce con il Gestore della memoria secondaria per operare sulla Memoria secondaria fisica.

Parallelamente a questa gerarchia, il Gestore delle transazioni coordina due moduli critici: il Gestore della concorrenza e il Gestore dell'affidabilità. Il Gestore della concorrenza interviene a livello dei metodi d'accesso per regolare l'ordine delle operazioni sui dati. Il Gestore dell'affidabilità monitora sia il Gestore del buffer sia il Gestore della memoria secondaria, assicurando che le modifiche vengano rese persistenti o annullate correttamente in caso di guasto.

1.2 Definizione di transazione

Una transazione può essere definita come un processo le cui azioni hanno effetti atomici: o vengono eseguite tutte o nessuna. Dal punto di vista della programmazione, essa è la parte di un programma che specifica tale processo. Una transazione ha un inizio, identificato dal comando **begin-transaction** (spesso implicito), e una fine, nota come **end-transaction** (anch'essa raramente esplicitata). All'interno di questo perimetro, deve essere eseguito una e una sola volta uno dei due comandi di chiusura: **commit [work]** per terminare correttamente consolidando le modifiche, o **rollback [work]** per abortire la transazione annullando ogni effetto prodotto.

Esiste una netta differenza tra un programma applicativo e una transazione: **un singolo programma può infatti contenere al suo interno più transazioni sequenziali**. Ad esempio, un programma potrebbe iniziare una transazione *T1*, eseguire delle azioni e terminarla con un **commit**, per poi iniziare immediatamente una transazione *T2*, eseguire altre azioni e concluderla con un altro **commit**.

1.2.1 Transazioni in JDBC

In JDBC, la gestione delle transazioni avviene tramite l'interfaccia `Connection` e il metodo `setAutoCommit(boolean autoCommit)`.

- Se impostato a `con.setAutoCommit(true)` (valore predefinito), il sistema opera in modalità “autocommit”: ogni singola operazione SQL viene trattata come una transazione autonoma che va immediatamente in commit.
- Se impostato a `con.setAutoCommit(false)`, la gestione è delegata al programma. Lo sviluppatore deve invocare esplicitamente `con.commit()` o `con.rollback()`. In JDBC non esiste un comando esplicito di `start transaction`; una nuova transazione inizia automaticamente all’apertura della connessione e subito dopo ogni commit o rollback della precedente.

1.2.2 Transazioni in SQL

Secondo lo standard SQL, una transazione inizia al primo comando SQL impartito dopo la connessione al database o dopo la conclusione della transazione precedente. Sebbene lo standard preveda il comando `START TRANSACTION`, esso non è obbligatorio e molti sistemi non lo implementano (sebbene sia presente in Postgres). La conclusione avviene tramite `commit [work]`, che rende permanenti le operazioni, o `rollback [work]`, che rinuncia alle esecuzioni riportando i dati allo stato iniziale. Anche in SQL è spesso presente la modalità autocommit.

Un esempio di transazione per il trasferimento di fondi tra conti è il seguente:

```
1 start transaction;
2 update ContoCorrente set Saldo = Saldo + 10 where NumConto = 12202;
3 update ContoCorrente set Saldo = Saldo - 10 where NumConto = 42177;
4 commit;
```

In scenari più complessi, la transazione può includere logica decisionale:

```
1 start transaction;
2 update ContoCorrente set Saldo = Saldo + 10 where NumConto = 12202;
3 update ContoCorrente set Saldo = Saldo - 10 where NumConto = 42177;
4 select Saldo into A from ContoCorrente where NumConto = 42177;
5 if (A >= 0) then commit else rollback;
```

1.3 Le proprietà ACID

Il concetto di transazione si fonda su quattro proprietà fondamentali, note con l’acronimo ACID:

1.3.1 Atomicità

La transazione è un’unità atomica indivisibile: tutto o niente. Se viene eseguito il commit, tutte le azioni hanno effetto; altrimenti, nessuna azione deve lasciare traccia. Non è ammesso alcuno stato intermedio. L’esito può essere un commit (conclusione naturale) o un abort (o rollback). L’abort può essere un “suicidio”, se richiesto esplicitamente dall’applicazione, o un “omicidio”, se imposto dal sistema a causa di crash, violazioni di vincoli, conflitti di concorrenza o incertezza nel fallimento. Il sistema gestisce internamente le operazioni di undo (annullamento) e redo (ripetizione) per garantire questa proprietà.

1.3.2 Consistenza

Una transazione deve rispettare i vincoli di integrità. Anche se tali vincoli possono essere temporaneamente violati “durante” lo svolgimento delle operazioni, è imperativo che sia lo stato iniziale che quello finale della base di dati siano corretti. Se lo stato finale viola i vincoli, la transazione fallisce. Normalmente i vincoli vengono verificati a ogni operazione, ma SQL permette di “differire” la verifica al momento del commit tramite la clausola `DEFERRABLE INITIALLY DEFERRED`.

I vincoli differiti sono necessari, ad esempio, in presenza di cicli di integrità referenziale. Si consideri il seguente schema Postgres:

```

1 create table r1 (a integer primary key, b integer, c integer);
2 create table r2 (d integer primary key, e integer, f integer);
3 alter table r1 add constraint r1_c_fk foreign key (c) references r2(d) deferrable
  initially deferred;
4 alter table r2 add constraint r2_c_fk foreign key (f) references r1(a) deferrable
  initially deferred;
5
6 start transaction;
7 insert into r1 values (3,3,3); -- Accettata grazie al differimento
8 insert into r2 values (3,3,3); -- Chiude il ciclo di riferimento
9 commit; -- Verifica finale: OK

```

Senza il differimento, l’inserimento in *r1* fallirebbe immediatamente perché la chiave 3 non esiste ancora in *r2*.

1.3.3 Isolamento

Una transazione non deve risentire degli effetti di altre transazioni eseguite contemporaneamente. L’esecuzione concorrente deve produrre un risultato equivalente a una qualche esecuzione sequenziale (serializzabilità). Inoltre, i risultati intermedi di una transazione non devono essere visibili alle altre; in caso contrario, si potrebbero utilizzare dati errati o attivare un “effetto domino” di annullamenti in caso di rollback. Il sistema protegge le operazioni bloccando le risorse necessarie finché la transazione non termina.

1.3.4 Durabilità (Persistenza)

Gli effetti di una transazione che ha effettuato con successo il commit devono essere permanenti e non possono andare perduti, nemmeno in caso di guasti successivi. Il termine “commit” implica un impegno formale del DBMS. È necessario prestare particolare attenzione alla gestione del buffer: in caso di crash, se i dati modificati non sono ancora stati scaricati (flushed) fisicamente su disco, il sistema deve essere in grado di recuperarli al riavvio.

1.4 Corrispondenza tra proprietà e moduli DBMS

Le proprietà ACID sono garantite da moduli specifici del sistema:

- **Atomicità e Durabilità:** sono affidate al Gestore dell’affidabilità.
- **Isolamento:** è garantito dal Gestore della concorrenza.
- **Consistenza:** è presidiata dal Gestore dell’integrità a tempo di esecuzione, con il supporto preventivo del compilatore del DDL (Data Definition Language).

2 Gestione dell’affidabilità: Atomicità e Durabilità

Il **Gestore della concorrenza** si interfaccia con i metodi d’accesso, e il **Gestore della affidabilità** monitora sia il buffer che la memoria secondaria per assicurare che ogni transazione sia atomica e i suoi effetti siano persistenti.

2.1 Persistenza delle memorie e tipologie di guasto

Il sistema si basa su diverse tipologie di supporto, caratterizzate da differenti livelli di persistenza. La **Memoria centrale** (RAM) è volatile e non persistente; il suo contenuto va perduto in caso di interruzione dell’energia. La **Memoria di massa** (disco) è persistente, ma soggetta a danneggiamenti fisici. Per ovviare a questi limiti, si introduce l’astrazione ideale di **Memoria stabile**, un supporto teoricamente indistruttibile. Nella realtà, tale astrazione viene perseguita attraverso la *ridondanza*, impiegando configurazioni come dischi replicati (RAID), nastri magnetici e altri sistemi di backup distribuiti.

La classificazione dei malfunzionamenti distingue tra tre categorie principali:

- **Guasti “soft”**: **includono errori di programma**, crash del sistema operativo o cadute di tensione. In queste circostanze, si perde il contenuto della memoria centrale, ma i dati nella memoria secondaria rimangono intatti. Il ripristino avviene tramite un **warm restart** (ripresa a caldo o recovery).
- **Guasti “hard”**: **colpiscono** direttamente i dispositivi di **memoria secondaria**, causandone la perdita parziale o totale. In questo caso, l'integrità è garantita solo dalla memoria stabile. Il ripristino richiede un **cold restart** (ripresa a freddo).
- **Catastrofe**: indica la **distruzione della memoria stabile**. Per fini progettuali, si assume solitamente che tale evento **non** possa verificarsi o che sia gestito tramite procedure di disaster recovery esterne al sistema ordinario.

2.2 Il modello “fail-stop”

Il comportamento del sistema durante un guasto è descritto dal **modello “fail-stop”**, rappresentabile come un grafo di stati. Nello stato **Normal**, il sistema esegue le operazioni ordinarie. In caso di errore (**Fail**), il sistema transita nello stato **Stop**. Dal momento dello stop, l'operazione di **Boot** porta il sistema nello stato di **Recovery**. Se il processo di ripristino ha successo (**Recovery completato**), si torna allo stato **Normal**. Qualora si verifichi un ulteriore guasto durante la fase di recovery, il sistema torna nuovamente nello stato **Stop**.

2.3 Il Gestore dell'affidabilità

Il **Gestore dell'affidabilità** ha il compito primario di assicurare le proprietà di **atomicità** (tutto o niente) e **durabilità** (persistenza degli effetti). Esso gestisce l'esecuzione dei comandi transazionali fondamentali: l'inizio della transazione (B , begin o start transaction), il consolidamento dei risultati (C , commit work) e l'annullamento delle operazioni (A , abort o rollback work). Inoltre, governa le operazioni di ripristino post-guasto attraverso le procedure di warm e cold restart.

2.4 Strumenti di ridondanza: Log e Dump

Per garantire il ripristino, il gestore utilizza informazioni ridondanti memorizzate in memoria stabile:

- **Log**: è un **archivio permanente in memoria stabile che registra sequenzialmente tutte le operazioni svolte**. Funge da traccia per poter tornare indietro o ricostruire le azioni.
- **Dump**: è una **copia di riserva completa della base di dati**, anch'essa conservata in memoria stabile. Rappresenta un “punto di partenza” statico da cui avviare la ricostruzione in caso di guasti hard.

2.5 Modello di riferimento e operazioni

Una transazione viene modellata come una sequenza di operazioni di input-output su oggetti astratti (blocchi o record). Le operazioni fondamentali sono:

- $I(O)$: inserimento di un nuovo oggetto O .
- $D(O)$: eliminazione (delete) di un oggetto O .
- $U(O)$: aggiornamento (update) di un oggetto O .

L'analisi prescinde dalla semantica applicativa di dettaglio, concentrandosi esclusivamente sui valori assunti dai dati. Si definisce **BS (before state)** il valore dell'oggetto prima dell'operazione (rilevante per eliminazioni e aggiornamenti) e **AS (after state)** il valore dell'oggetto dopo l'operazione (rilevante per inserimenti e aggiornamenti). Il sistema registra, ad esempio, che un valore è passato da 3 a 4, ignorando la logica di incremento che ha generato il cambiamento.

2.6 Il Log: Struttura e regole di scrittura

Il log ospita diverse tipologie di record, identificati dalla transazione T , dall'oggetto O e dai valori di stato:

- **Begin**: $B(T)$
- **Insert**: $I(T, O, AS)$
- **Delete**: $D(T, O, BS)$
- **Update**: $U(T, O, BS, AS)$
- **Commit**: $C(T)$

- **Abort:** $A(T)$
- **Record di sistema:** includono i riferimenti ai **dump** e ai **checkpoint**.

La conclusione positiva di una transazione avviene solo quando il record di **commit** viene scritto nel log. Tale operazione deve essere **irrevocabile**, pertanto il **commit** va scritto immediatamente in memoria stabile tramite un'operazione di **force**. Al contrario, i *record di abort* possono essere scritti in modalità *asincrona*. Se avviene un guasto prima del commit, lo stato originario della base di dati deve essere ricostruito; se avviene dopo, lo stato finale deve essere consolidato.

Per manipolare gli stati, si definiscono due meccanismi fondamentali:

- **Undo:** annulla un'azione su un oggetto O . Per aggiornamenti o eliminazioni, copia il BS nell'oggetto O ; per gli inserimenti, elimina l'oggetto O se presente.
- **Redo:** ripete un'azione su un oggetto O . Per inserimenti o aggiornamenti, copia l' AS nell'oggetto O ; per le eliminazioni, rimuove l'oggetto O se presente.

NB: Entrambe le operazioni sono caratterizzate dalla proprietà di **idempotenza**: eseguire più volte consecutivamente la stessa operazione produce il medesimo effetto di un'esecuzione singola ($undo(undo(A)) = undo(A)$ e $redo(redo(A)) = redo(A)$).

2.6.1 Regole fondamentali di gestione del log

La corretta gestione del log segue il principio secondo cui la nota scritta deve precedere l'azione fisica:

1. **Write-Ahead-Log (WAL):** è necessario scrivere il BS sul log in memoria stabile prima che il valore corrispondente venga scritto fisicamente nella base di dati. Ciò garantisce la possibilità di “disfare” (undo) le azioni.
2. **Commit-Precedenza:** è necessario scrivere l' AS sul log in memoria stabile prima di effettuare il **commit** della transazione. Ciò garantisce la possibilità di “rifare” (redo) le azioni.

Esistono tre modalità di coordinamento tra le scritture nel log e nella base di dati:

- **Modalità immediata:** le scritture nel DB possono avvenire **prima del commit**. Il DB contiene valori modificati sia da transazioni completate che da transazioni ancora in corso. In caso di crash, le transazioni attive richiedono un'operazione di **UNDO-only (No-REDO)**.
- **Modalità differita:** le scritture nel DB avvengono **solo dopo il commit**. Il DB non contiene mai valori di transazioni non ancora terminate, ma non è garantito che contenga i nuovi valori di quelle completate. In caso di crash, le transazioni concluse richiedono un'operazione di **REDO-only (No-UNDO)**.
- **Modalità mista:** permette scritture sia immediate che differite. È la modalità più **efficiente** poiché consente al gestore del buffer di decidere quando scrivere sul disco in base al carico operativo. In caso di guasto, richiede la gestione completa di **REDO** e **UNDO**.

2.7 La procedura di Rollback

Il **rollback** è l'operazione di annullamento di una transazione che non è giunta al commit. La procedura prevede di scandire il log a **ritroso**, dalla fine verso l'inizio, fino a incontrare il record di *begin* della transazione T interessata. Per ogni record incontrato che appartiene a T , il sistema esegue l'operazione di **undo**. Al termine, viene aggiunto in coda al log un record di **rollback** per segnalare che l'annullamento è stato completato correttamente. Si procede a ritroso per due motivi: le azioni da annullare sono generalmente vicine alla fine del log e la scansione inversa assicura che, in caso di azioni multiple sullo stesso oggetto, venga correttamente ripristinato il valore iniziale originario.

3 Gestione dell'affidabilità: Procedure di Recovery e Ripristino

Il concetto fondamentale alla base della gestione dell'affidabilità in un sistema di basi di dati è il **recovery**, noto anche come ripristino o **warm restart**. Lo scopo primario di questa funzione è assicurare che il contenuto della base di dati rimanga in uno stato corretto e coerente sia a seguito di un guasto imprevisto, sia durante la procedura ordinaria di spegnimento (**shut down**) del sistema. In particolare, al momento dello spegnimento, è imperativo che lo stato del database non rifletta operazioni derivanti da transazioni rimaste incompiute; parallelamente, tutte le transazioni che hanno effettuato con successo il commit devono essere state effettivamente completate e rese persistenti.

Poiché la conclusione di una sessione operativa non è sempre soggetta a un controllo totale — si pensi a un arresto anomalo o a un crash improvviso del sistema — si rende necessaria una verifica sistematica a ogni avvio dello strumento. Si potrebbe ipotizzare un approccio ingenuo e inefficiente per implementare tale controllo: esaminare ogni singola transazione registrata e, se questa risulta conclusa con un commit nel log, eseguire nuovamente tutte le sue azioni (**redo**) per garantirne la persistenza su disco; se la transazione è stata annullata tramite **rollback**, non si intraprende alcuna azione; in tutti gli altri casi, si procede a disfare ogni operazione effettuata (**undo**) direttamente sul supporto fisico.

3.1 L'algoritmo di Recovery

Per ovviare alle inefficienze del metodo ingenuo, si adotta un algoritmo strutturato che identifica con precisione quali azioni debbano essere trattate. La logica si basa sulla creazione di due insiemi distinti: l'insieme *UNDO*, contenente le transazioni da disfare poiché interrotte o fallite, e l'insieme *REDO*, contenente le transazioni da rifare poiché confermate ma potenzialmente non ancora stabili su disco.

L'algoritmo si articola nei seguenti passaggi:

1. **Identificazione e Fase di UNDO:** Si percorre il log all'indietro, partendo dalla fine verso l'inizio. Per ogni record incontrato:
 - Se il record indica un **commit**, la transazione viene aggiunta all'insieme *REDO*.
 - Se il record indica un **rollback**, la transazione viene aggiunta a un insieme di supporto denominato *RB*.
 - Se si incontra un'azione compiuta da una transazione che non appartiene né a *REDO* né a *RB*, tale azione viene immediatamente disfatta (**undo**).
2. **Fase di REDO:** Si percorre il log in avanti, dall'inizio verso la fine. Per ogni record che descrive un'azione appartenente a una transazione inclusa nell'insieme *REDO*, tale azione viene rieseguita (**redo**).

Le apparenti incoerenze che potrebbero sorgere tra queste due fasi, dovute a transazioni diverse operanti simultaneamente sui medesimi dati, sono prevenute ed evitate dai meccanismi di controllo della concorrenza integrati nel DBMS.

3.2 Ottimizzazione tramite Checkpoint

L'esecuzione del recovery standard è onerosa, poiché richiede la doppia lettura integrale del log e la manipolazione di una mole massiccia di operazioni. Per incrementare l'efficienza, è necessario limitare la profondità della scansione a ritroso del log, agendo sulla certezza che, oltre un determinato punto, non vi siano buffer non salvati o transazioni lasciate in sospeso. Questo obiettivo viene perseguito tramite la tecnica del **checkpoint**.

3.2.1 Checkpoint quiescente (inattivo)

Il checkpoint quiescente rappresenta una “chiusura dei conti” periodica, paragonabile alla chiusura amministrativa di fine anno in cui si sospende l'accettazione di nuove pratiche per evadere quelle correnti. La procedura segue passi rigidi:

- Si interrompe l'accettazione di nuove transazioni.
- Si attende che tutte le transazioni attualmente attive giungano a conclusione.
- Si esegue la scrittura forzata (**force**) su disco di tutte le pagine “sporche” presenti nel buffer di memoria.
- Si inserisce un record di checkpoint nel log e se ne forza la scrittura immediata su disco.
- Si riprende il normale funzionamento accettando nuove transazioni.

Con questa strategia, il recovery non deve più analizzare il log dall'inizio, ma può fermarsi all'ultimo record di checkpoint incontrato procedendo a ritroso. È prassi eseguire un checkpoint allo startup e allo shutdown per semplificare i ripristini futuri. Tuttavia, questa modalità introduce inefficienze operative dovute ai tempi morti necessari per attendere la fine delle transazioni in corso.

3.2.2 Checkpoint non quiescente

Il checkpoint non quiescente mira a semplificare il ripristino registrando quali transazioni sono attive in un istante preciso senza bloccare l'operatività del sistema. La procedura prevede di interrompere brevemente l'accettazione di ogni tipo di richiesta (nuove transazioni, scritture, commit, abort) per rilevare l'elenco delle transazioni correnti. Successivamente, si forzano su disco tutte le pagine sporche (o solo quelle delle transazioni già confermate, a seconda della strategia di gestione del buffer adottata). Infine, si scrive nel log un record di checkpoint contenente gli identificativi delle transazioni attive rilevate e si effettua il force prima di riprendere le attività.

L'algoritmo di recovery associato è più articolato:

1. **Fase 1:** Si percorre il log all'indietro fino al checkpoint per costruire gli insiemi *REDO*, *RB* e disfare le azioni di *UNDO*.
2. **Fase 1 bis:** Si prosegue la scansione a ritroso dal checkpoint fino a raggiungere il record di *begin* della transazione più vecchia tra quelle elencate come attive nel checkpoint stesso. In questo tratto, l'unica operazione rilevante è continuare a disfare le azioni delle transazioni che non risultano in *REDO*.
3. **Fase 2:** Si ripercorre il log in avanti partendo dal punto limite raggiunto nella fase 1 bis (o dal checkpoint, se questo garantiva il salvataggio di tutti i buffer sporchi), rifacendo le azioni delle transazioni in *REDO*.

3.3 Esempio di ripresa a caldo

Si analizzi un esempio pratico di ripresa a caldo basato sulla seguente sequenza di operazioni registrate nel log: $B(T1), B(T2), U(T2, O1, B1, A1), I(T1, O2, A2), B(T3), C(T1), B(T4), U(T3, O2, B3, A3), U(T4, O3, B4, A4), CK(T2, T3, T4), C(T4), B(T5), U(T3, O3, B5, A5), U(T5, O4, B6, A6), D(T3, O5, B7), A(T3), C(T5), I(T2, O6, A8) \rightarrow Crash$

Un diagramma temporale aiuta a visualizzare la situazione:

- $T1$ inizia e termina con commit prima del checkpoint.
- $T2$ inizia prima del checkpoint e rimane attiva fino al crash.
- $T3$ inizia prima del checkpoint e termina con abort prima del crash.
- $T4$ inizia prima del checkpoint e termina con commit dopo di esso.
- $T5$ inizia dopo il checkpoint e termina con commit prima del crash.

3.3.1 Evoluzione dell'algoritmo (Fase 1 - Scansione a ritroso)

1. Inizialmente $UNDO = \emptyset, REDO = \emptyset$.
2. Si incontra $I(T2, O6, A8)$: $T2$ non è in *REDO*, quindi si esegue l'undo dell'azione eliminando l'oggetto $O6$. $UNDO = T2$.
3. Si incontra $C(T5)$: $T5$ viene inserito in *REDO*.
4. Si incontra $A(T3)$: $T3$ viene gestita come transazione da disfare (assumendo che gli abort siano registrati senza rollback immediato).
5. Si incontra $D(T3, O5, B7)$: si ripristina $O5 = B7$.
6. Si incontra $U(T5, O4, B6, A6)$: non si effettua nulla (fase undo).
7. Si incontra $U(T3, O3, B5, A5)$: si ripristina $O3 = B5$.
8. Si incontra $C(T4)$: $T4$ entra in *REDO*. $UNDO = T2, T3$.
9. Si raggiunge il checkpoint $CK(T2, T3, T4)$. Si prosegue a ritroso fino all'inizio della più vecchia ($T2$).
10. Si incontra $U(T3, O2, B3, A3)$: si ripristina $O2 = B3$.
11. Si incontra $U(T2, O1, B1, A1)$: si ripristina $O1 = B1$.

3.3.2 Evoluzione dell'algoritmo (Fase 2 - Scansione in avanti)

Si rifanno le azioni solo per $REDO = T4, T5$:

- Si rifà $U(T4, O3, B4, A4) \rightarrow O3 = A4$.
- Si rifà $U(T5, O4, B6, A6) \rightarrow O4 = A6$.

3.4 Tipologie di guasti e procedure di backup

I guasti che colpiscono un sistema informativo si dividono in due categorie principali:

- **Guasti “soft”**: causano la perdita del contenuto della memoria centrale ma non danneggiano la memoria secondaria. Il ripristino avviene tramite la ripresa a caldo (**warm restart** o *recovery*).
- **Guasti “hard”**: colpiscono direttamente i dispositivi di memoria secondaria. Causano la perdita fisica dei dati sul disco, ma non colpiscono la memoria stabile (dove risiedono log e backup). Il ripristino richiede la ripresa a freddo (**cold restart**).

La ripresa a freddo si appoggia al **Dump**, una copia completa di riserva (backup) della base di dati. Il dump viene solitamente prodotto a sistema fermo, salvato in memoria stabile e registrato nel log con un record apposito che specifica l'istante di esecuzione e i dettagli tecnici. Poiché l'operazione è onerosa, il dump viene eseguito con una frequenza molto minore rispetto al checkpoint. La procedura di cold restart prevede:

1. Ripristino integrale dei dati a partire dall'ultimo backup disponibile.
2. Esecuzione sequenziale delle operazioni registrate nel log dall'istante del dump fino al momento del guasto.
3. Esecuzione finale di una ripresa a caldo per gestire correttamente le transazioni attive al momento dell'interruzione.

4 Controllo della concorrenza

La gestione della concorrenza è un requisito fondamentale per i moderni sistemi informativi. Ambienti critici come gli istituti bancari, i sistemi di prenotazione aerea o i social network gestiscono decine o centinaia di transazioni al secondo. In tali contesti, l'esecuzione seriale — ovvero l'esecuzione di ogni transazione per intero senza alcuna interposizione di azioni esterne — risulterebbe inaccettabile a causa dei tempi di attesa proibitivi. Tuttavia, l'intercalamento indiscriminato delle azioni genera anomalie che devono essere governate dal DBMS per preservare la coerenza dei dati.

4.1 La perdita di aggiornamento (Lost Update)

L'anomalia della perdita di aggiornamento si verifica tipicamente in scenari di scrittura-scrittura ($W-W$). Si consideri il caso di due prelevamenti simultanei da un medesimo conto bancario con saldo iniziale $x = 1000$. Due transazioni, t_1 e t_2 , operano seguendo la logica di lettura, modifica e riscrittura:

- $t_1 : r(x), x = x - 200, w(x)$
- $t_2 : r(x), x = x - 100, w(x)$

In un'esecuzione seriale (tutta t_1 seguita da tutta t_2), il saldo finale corretto sarebbe $x = 700$. Tuttavia, in un'esecuzione concorrente non governata, può verificarsi la seguente sequenza temporale:

1. t_1 esegue $r_1(x)$, leggendo 1000.
2. t_1 calcola internamente $x = x - 200$ (risultato locale 800).
3. t_2 esegue $r_2(x)$, leggendo ancora 1000 poiché t_1 non ha ancora scritto.
4. t_2 calcola internamente $x = x - 100$ (risultato locale 900).
5. t_1 esegue $w_1(x)$, scrivendo 800 e termina con il *commit*.
6. t_2 esegue $w_2(x)$, scrivendo 900 e termina con il *commit*.

Al termine di questa sequenza, il saldo finale registrato è 900. L'aggiornamento effettuato da t_1 viene completamente perso (sovrascritto da t_2), portando a uno stato della base di dati incoerente rispetto alle operazioni realmente avvenute. Le azioni si sono alternate in modo inaccettabile, laddove un'esecuzione seriale avrebbe garantito la correttezza del risultato.

4.2 La lettura sporca (Dirty Read)

L'anomalia della lettura sporca si manifesta quando una transazione legge un dato modificato da un'altra transazione che non ha ancora confermato l'operazione (mancanza di *commit*). Si consideri un conto con saldo 1000. La transazione t_1 riceve un assegno da 10.000 e procede all'aggiornamento:

- $t_1 : r_1(x), x = x + 11000, w_1(x)$

Prima che t_1 possa confermare l'operazione, la transazione t_2 esegue una lettura:

- $t_2 : r_2(x)$

In questo istante, t_2 legge un saldo di 11.000 e potrebbe comunicare all'esterno questo stato (ad esempio confermando all'utente la disponibilità dei fondi). Tuttavia, se l'assegno di t_1 risulta falso, t_1 deve eseguire un *abort* per annullare l'operazione, riportando il saldo a 1000. La transazione t_2 ha dunque operato su uno stato intermedio "sporco" che non è mai diventato definitivo nella base di dati, leggendo un dato errato.

4.3 Letture inconsistenti e aggiornamento fantasma

Le letture inconsistenti si verificano quando una transazione legge dati correlati che vengono modificati nel frattempo da un'altra transazione. Si consideri uno scenario con due conti, $y = 1000$ e $z = 1000$ (somma totale 2000). La transazione t_2 sposta 500 da y a z : $t_2 : r_2(y), y = y - 500, r_2(z), z = z + 500, w_2(y), w_2(z), commit$

Se la transazione t_1 tenta di calcolare la somma totale $s = y + z$ in modo concorrente, potrebbe verificarsi la seguente sequenza:

1. t_1 esegue $r_1(y)$ e legge 1000.
2. t_2 esegue tutte le sue operazioni di spostamento e termina con il *commit*. Ora $y = 500$ e $z = 1500$.
3. t_1 esegue $r_1(z)$ e legge 1500.
4. t_1 calcola $s = y + z = 1000 + 1500 = 2500$.

Il valore $s = 2500$ è errato, poiché la somma dei due conti non è mai stata realmente pari a tale cifra. La transazione t_1 ha visto uno stato non esistente o non coerente della base di dati.

Un caso ancora più elementare di lettura inconsistente si verifica quando t_1 legge due volte lo stesso oggetto x :

1. t_1 esegue $r_1(x)$.
2. t_2 esegue $r_2(x), x = x + 1, w_2(x)$ e *commit*.
3. t_1 esegue nuovamente $r_1(x)$. In questa circostanza, t_1 legge due valori diversi per lo stesso oggetto all'interno della medesima transazione. Questo problema può verificarsi tipicamente durante l'esecuzione di un join con l'algoritmo nested-loop, dove la medesima relazione viene letta ripetutamente.

4.4 L'inserimento fantasma (Phantom)

L'inserimento fantasma riguarda l'apparizione di nuovi dati che soddisfano una condizione di ricerca durante l'esecuzione di una transazione. Si consideri un sistema di prenotazione aerea:

1. t_1 esegue un'operazione per "contare i posti disponibili".
2. t_2 inserisce nuovi record nel database, ad esempio perché l'aereo è stato sostituito con uno più grande, e termina con il *commit*.
3. t_1 esegue nuovamente il conteggio dei posti disponibili.

Sebbene t_1 non abbia subito modifiche dirette sui record precedentemente letti, il risultato finale del conteggio è cambiato. L'anomalia risiede nel fatto che t_2 ha inserito un dato "nuovo" che interferisce con la visione coerente che t_1 dovrebbe avere della base di dati.

4.5 Sintesi delle anomalie di concorrenza

Le anomalie generate dall'interazione tra letture (R) e scritture (W) possono essere riassunte come segue:

- **Perdita di aggiornamento:** Conflitto di tipo Scrittura-Scrittura ($W - W$).
- **Lettura sporca:** Conflitto Lettura-Scrittura ($R - W$) o $W - W$ in presenza di un *abort*.
- **Letture inconsistenti:** Conflitto $R - W$ su dati esistenti.
- **Aggiornamento fantasma:** Conflitto $R - W$ dove i dati letti non sono più coerenti con lo stato della base di dati.
- **Inserimento fantasma:** Conflitto $R - W$ che coinvolge l'inserimento di dati precedentemente non esistenti.

5 Teoria e pratica del controllo della concorrenza nelle basi di dati

Il controllo della concorrenza è una componente essenziale dei moderni sistemi di gestione di basi di dati, necessaria per garantire che l'esecuzione simultanea di più transazioni non comprometta l'integrità delle informazioni. Senza un'adeguata regolamentazione, l'intercalamento delle operazioni di lettura e scrittura può generare diverse tipologie di anomalie. ## Livelli di isolamento in SQL:1999 e JDBC

Per bilanciare la necessità di consistenza con le prestazioni del sistema, lo standard SQL:1999 e l'interfaccia JDBC permettono di definire il comportamento transazionale attraverso i livelli di isolamento. Una transazione può essere dichiarata **read-only**, precludendo ogni operazione di scrittura. Per le transazioni che operano in lettura e scrittura, è possibile selezionare uno dei quattro livelli standardizzati, ognuno dei quali offre una protezione progressiva contro le anomalie:

1. **Read Uncommitted**: è il livello più permissivo; consente l'occorrenza di letture sporche, letture inconsistenti, aggiornamenti fantasma e inserimenti fantasma.
2. **Read Committed**: garantisce l'assenza di letture sporche, ma permette ancora letture inconsistenti, aggiornamenti fantasma e inserimenti fantasma.
3. **Repeatable Read**: evita tutte le anomalie precedentemente citate, ad eccezione degli inserimenti fantasma.
4. **Serializable**: è il livello di massima sicurezza che previene ogni tipologia di anomalia, garantendo che l'effetto dell'esecuzione concorrente sia identico a una qualche esecuzione seriale.

È fondamentale osservare che, sebbene la perdita di aggiornamento dovrebbe teoricamente essere sempre evitata, nella pratica i DBMS richiedono l'uso del livello **serializable** per proteggere con certezza le transazioni che effettuano scritture. La scelta di livelli di isolamento inferiori è giustificata dal fatto che la gestione rigorosa della concorrenza è computazionalmente costosa; se l'applicazione può tollerare una precisione approssimativa (specialmente nelle letture), si può rinunciare a parte dell'isolamento per guadagnare in efficienza.

5.1 Architettura del Gestore della Concorrenza

L'implementazione fisica del controllo della concorrenza avviene attraverso un modulo dedicato denominato **Gestore della concorrenza**. All'interno dell'architettura del DBMS, questo componente si colloca tra il **Gestore dei metodi d'accesso** e il **Gestore della memoria secondaria**. Il flusso operativo prevede che il Gestore delle transazioni invii segnali di *begin*, *commit*, *abort*. Contemporaneamente, il Gestore dei metodi d'accesso invia richieste di *read* e *write*. Il Gestore della concorrenza riceve queste richieste e, consultando una **Tabella dei lock**, decide se autorizzarle, rifiutarle o riordinarle prima di inoltrarle al Gestore della memoria secondaria per l'accesso effettivo alla base di dati. Questo coordinamento avviene ignorando, in questa fase di astrazione, le problematiche relative ai buffer e all'affidabilità.

5.2 Teoria del controllo di concorrenza: Schedule e Serializzabilità

Dal punto di vista teorico, una transazione è modellata come una sequenza di operazioni di lettura (*r*) e scrittura (*w*), identificata da un numero univoco e conclusa da un commit (*c*) o un abort (*a*). Ad esempio: $t_1 : r_1(x)r_1(y)w_1(x)w_1(y)c_1$. Uno **schedule** *S* è una sequenza di operazioni di input/output provenienti da diverse transazioni. Per semplicità analitica, inizialmente si considerano solo le transazioni che effettuano il commit, ignorando quelle che vanno in abort; questa astrazione è definita **commit-proiezione dello schedule reale**.

L'obiettivo principale è evitare le anomalie. Uno **schedule seriale** è una sequenza in cui le transazioni sono eseguite integralmente una dopo l'altra, senza alcuna interposizione (es. $S_2 : r_0(x)r_0(y)w_0(x)r_1(y)r_1(x)w_1(y)r_2(x)r_2(y)r_2(z)w_2(z)$). Gli schedule seriali sono intrinsecamente privi di anomalie. Uno schedule è definito **serializzabile** se produce lo stesso risultato di uno schedule seriale operante sulle medesime transazioni. Lo **Scheduler** ha il compito di ammettere solo schedule serializzabili, individuando classi di schedule la cui serializzabilità sia verificabile rapidamente.

5.2.1 View-Serializzabilità

La view-serializzabilità si basa sul concetto di equivalenza di “visione” dei dati. Per definirla, occorrono due nozioni preliminari:

- **Scrittura finale:** un'operazione $w_i(x)$ è una scrittura finale in uno schedule S se è l'ultima operazione di scrittura eseguita sull'oggetto x in quello schedule. Determina lo stato finale della base di dati.
- **Relazione Legge-da:** un'operazione $r_i(x)$ legge-da $w_j(x)$ in uno schedule S se $w_j(x)$ precede $r_i(x)$ e non vi sono altre scritture $w_k(x)$ interposte tra le due. Determina l'influenza che una transazione ha sulle altre.

Due schedule S_i e S_j sono **view-equivalenti** ($S_i \approx_V S_j$) se presentano le stesse scritture finali e la medesima relazione legge-da. Uno schedule è **view-serializzabile** (VSR) se esiste uno schedule seriale view-equivalente ad esso. Ad esempio, dati gli schedule: $S_2 : r_2(x)w_0(x)r_1(x)w_2(x)w_2(z)$ $S_3 : w_0(x)r_2(x)r_1(x)w_2(x)w_2(z)$ I due non sono view-equivalenti perché in S_2 la lettura $r_1(x)$ legge da $w_0(x)$, mentre in S_3 sia $r_2(x)$ che $r_1(x)$ leggono da $w_0(x)$. Se esiste uno schedule seriale $S_4 : w_0(x)r_1(x)r_2(x)w_2(x)w_2(z)$ che è view-equivalente a S_3 , allora S_3 è view-serializzabile.

La verifica della view-serializzabilità è tuttavia un problema **NP-completo**, il che la rende inutilizzabile nella pratica ingegneristica dei DBMS.

5.3 Conflict-Serializzabilità

Poiché trovare tutti gli schedule serializzabili è troppo costoso, si ricerca un sottoinsieme più facile da verificare. La **conflict-serializzabilità** (CSR) si basa sui conflitti tra operazioni. Un'azione a_i è in **conflitto** con a_j se appartengono a transazioni diverse ($i \neq j$), operano sul medesimo oggetto e almeno una delle due è una scrittura. Si distinguono conflitti **read-write** (rw o wr) e conflitti **write-write** (ww).

Due schedule S_i e S_j sono **conflict-equivalenti** ($S_i \approx_C S_j$) se ogni coppia di operazioni in conflitto mantiene il medesimo ordine in entrambi gli schedule. Uno schedule è conflict-serializzabile se esiste uno schedule seriale conflict-equivalente ad esso.

5.3.1 Relazione tra CSR e VSR

Esiste un teorema fondamentale: **ogni schedule conflict-serializzabile è anche view-serializzabile, ma non vale necessariamente il viceversa**. Un controesempio che dimostra la non necessità è lo schedule $r_1(x) \ w_2(x) \ w_1(x) \ w_3(x)$. Esso è view-serializzabile (essendo view-equivalente allo schedule seriale $r_1(x) \ w_1(x) \ w_2(x) \ w_3(x)$ poiché hanno legge-da vuota e la stessa scrittura finale $w_3(x)$), ma non è conflict-serializzabile a causa dei conflitti tra $w_2(x)$ e $w_1(x)$ che impediscono di riportarlo a un ordine seriale senza invertire una coppia in conflitto.

L'idea fondamentale della gestione pratica della concorrenza consiste nel rinunciare ad alcuni schedule serializzabili (quelli che sono VSR ma non CSR) per poter decidere rapidamente attraverso tecniche come il **Locking a due fasi (2PL)** o i protocolli **multiversione**.

6 Relazioni tra Conflict-Serializzabilità e View-Serializzabilità

Per quanto riguarda la sufficienza, ovvero la dimostrazione che CSR implica VSR, è necessario provare che la conflict-equivalenza ($\approx_C$) implica la view-equivalenza ($\approx_V$). Siano $S_1 \approx_C S_2$ due schedule. Essi devono presentare le medesime scritture finali; se così non fosse, esisterebbero due operazioni di scrittura sul medesimo oggetto in ordine diverso, il che costituirebbe un conflitto $W - W$ non rispettato, violando l'ipotesi di conflict-equivalenza. Allo stesso modo, devono presentare la stessa relazione “legge-da”: una discrepanza in tale relazione implicherebbe un ordine diverso tra scritture o tra coppie lettura-scrittura sullo stesso oggetto, violando nuovamente la conflict-equivalenza.

L'organizzazione strutturale degli schedule può essere visualizzata come un diagramma a ellissi concentriche. Al centro si trovano gli **Schedule Seriali**. Questi sono contenuti nell'insieme degli **Schedule S2PL** (Strict Two-Phase Locking), i quali sono a loro volta un sottoinsieme degli **Schedule 2PL**. Procedendo verso l'esterno, troviamo l'insieme degli **Schedule CSR**, contenuti negli **Schedule VSR**, fino a comprendere l'universo di tutti gli **Schedule** possibili.

6.1 Verifica della Conflict-Serializzabilità tramite Grafi

Lo strumento principale per verificare se uno schedule appartiene alla classe CSR è il **grafo dei conflitti**. Si tratta di un grafo orientato costruito secondo le seguenti regole:

- Viene creato un nodo per ogni transazione t_i presente nello schedule.
- Viene tracciato un arco orientato da t_i a t_j se sussiste almeno un conflitto tra un'azione a_i e un'azione a_j tale che a_i precede cronologicamente a_j nello schedule.

Il teorema cardine della verifica stabilisce che **uno schedule S è in CSR se e solo se il suo grafo dei conflitti è aciclico**. La dimostrazione si basa su un lemma il quale afferma che due schedule conflict-equivalenti possiedono necessariamente il medesimo grafo dei conflitti. Se S è in CSR, esso è conflict-equivalente a uno schedule seriale S_0 . Poiché il grafo di uno schedule seriale è intrinsecamente aciclico (i conflitti tra azioni a_i e a_j possono avvenire solo se $i < j$, rendendo impossibile la chiusura di un ciclo che richiederebbe un arco $i > j$), allora anche il grafo di S deve essere aciclico. Viceversa, se il grafo è aciclico, è possibile determinare un **ordinamento topologico** dei nodi (una numerazione tale che esistano solo archi (i, j) con $i < j$). Lo schedule seriale ottenuto ordinando le transazioni secondo tale sequenza sarà equivalente a S , provando la sua appartenenza a CSR.

Si consideri l'esempio di un grafo con sei transazioni dove gli archi sono: $4 \rightarrow 1$, $4 \rightarrow 2$, $1 \rightarrow 2$, $1 \rightarrow 5$, $5 \rightarrow 2$, $5 \rightarrow 3$, $5 \rightarrow 6$ e $2 \rightarrow 3$. Un possibile ordinamento topologico per questo grafo è la sequenza 4, 1, 5, 3, 2, 6. Questo ordinamento garantisce che per ogni conflitto (i, j) individuato nel grafo, la transazione i preceda sempre la transazione j nello schedule seriale equivalente.

6.2 Esempio di Grafo dei Conflitti

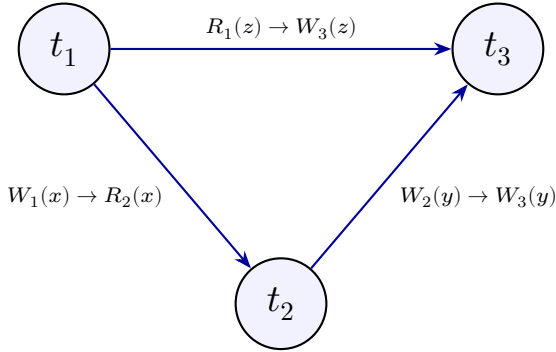


Figura A: Grafo Aciclico (CSR)

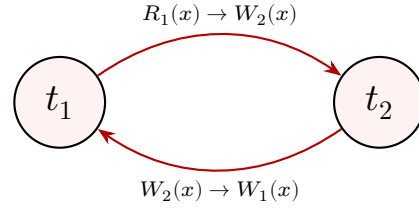


Figura B: Grafo Ciclico (no CSR)

L'illustrazione sopra mostra come verificare se uno schedule è *Conflict-Serializzabile* (CSR) utilizzando il grafo delle precedenze (o dei conflitti). Un principio fondamentale stabilisce che uno schedule è corretto (serializzabile) se e solo se il suo grafo è privo di cicli.

- **Figura A (Grafo Aciclico):** Consideriamo lo schedule $S_A : W_1(x)R_1(z)R_2(x)W_2(y)W_3(z)W_3(y)$. Il grafo viene costruito analizzando le operazioni in conflitto temporale:
 - L'operazione $W_1(x)$ precede $R_2(x)$, creando un conflitto Scrittura-Lettura (W-R) e definendo l'arco $t_1 \rightarrow t_2$.
 - L'operazione $W_2(y)$ precede $W_3(y)$, creando un conflitto Scrittura-Scrittura (W-W) sulla stessa risorsa, aggiungendo l'arco $t_2 \rightarrow t_3$.
 - L'operazione $R_1(z)$ precede $W_3(z)$, creando un conflitto Lettura-Scrittura (R-W) e tracciando l'arco $t_1 \rightarrow t_3$.

Il risultato è un grafo **aciclico** (DAG). Da questo grafo è possibile estrarre un *ordinamento topologico* lineare valido. Lo scheduler può quindi confermare che S_A è equivalente allo schedule seriale $t_1 \rightarrow t_2 \rightarrow t_3$. Lo schedule rientra nella classe CSR e le transazioni possono procedere al commit.

- **Figura B (Grafo con Ciclo):** Consideriamo lo schedule problematico $S_B : R_1(x)W_2(x)W_1(x)$. In questo caso, il grafo evidenzia un'anomalia:
 - $R_1(x)$ precede $W_2(x)$, creando un conflitto Lettura-Scrittura (R-W) e generando l'arco $t_1 \rightarrow t_2$.
 - Successivamente, t_1 esegue $W_1(x)$ dopo che t_2 ha eseguito $W_2(x)$. Questo crea un conflitto Scrittura-Scrittura (W-W) in direzione opposta, generando l'arco $t_2 \rightarrow t_1$.

Si forma così un **ciclo** ($t_1 \nrightarrow t_2$). Questo indica un'impossibilità logica: due transazioni non possono precedersi a vicenda in un ordine seriale. L'algoritmo (ad esempio tramite visita DFS) rileverà il ciclo; lo scheduler stabilirà quindi che S_B non è CSR ed eseguirà un abort per preservare l'isolamento e la consistenza del database.

6.3 Meccanismi di Locking

Nella pratica, la conflict-serializzabilità è verificabile più rapidamente rispetto alla view-serializzabilità (complessità lineare con strutture dati opportune). Tuttavia, uno scheduler reale deve operare in modo incrementale ad ogni richiesta di operazione, rendendo impraticabile il mantenimento e la verifica costante dell'aciclicità del grafo. Si adottano quindi tecniche che garantiscono la CSR senza costruire il grafo, come il **Locking a due fasi (2PL)** e il **timestamp** (spesso con implementazioni multiversione).

Il principio del locking prevede che l'accesso ai dati sia controllato per prevenire interferenze:

- Le operazioni di scrittura e lettura sono incompatibili tra loro sullo stesso oggetto.
- Diverse operazioni di lettura sono compatibili e possono coesistere.
- Ogni lettura deve essere preceduta da un **r_lock** (lock condiviso) e seguita da un **unlock**.
- Ogni scrittura deve essere preceduta da un **w_lock** (lock esclusivo) e seguita da un **unlock**.
- In caso di passaggio da lettura a scrittura sullo stesso oggetto, la transazione può richiedere un lock esclusivo immediato o effettuare un **lock upgrade** (da condiviso a esclusivo).

La gestione è affidata al **lock manager**, che utilizza una **tavola dei conflitti** per accogliere o rifiutare le richieste. Se una risorsa è libera (stato *free*), una richiesta di **r_lock** o **w_lock** viene accolta portando la risorsa rispettivamente in stato **r_locked** o **w_locked**. Se la risorsa è già **r_locked**, una nuova richiesta di **r_lock** è accettata (incrementando un contatore di lettori), mentre una di **w_lock** viene rifiutata. Se la risorsa è **w_locked**, ogni ulteriore richiesta di lock viene negata.

6.3.1 Il Locking a Due Fasi (2PL)

L'uso dei lock da solo non è sufficiente a prevenire anomalie come la perdita di aggiornamento. Il protocollo **2PL** introduce un vincolo temporale: una transazione, dopo aver rilasciato il suo primo lock (fase di rilascio o *shrinking*), non può acquisirne altri. Questo divide la vita della transazione in due fasi:

1. **Fase di acquisizione:** la transazione ottiene tutti i lock necessari senza rilasciarne alcuno.
2. **Fase di rilascio:** la transazione rilascia i lock e non può più richiederne di nuovi.

Il protocollo 2PL garantisce la conflict-serializzabilità. Per dimostrare che 2PL implica CSR, si consideri per ogni transazione l'istante in cui possiede tutti i lock ed è in procinto di rilasciare il primo. Ordinando le transazioni secondo questi istanti, si ottiene uno schedule seriale S^* che è conflict-equivalente allo schedule originale. Tuttavia, non tutti gli schedule CSR sono prodotti dal 2PL. Un controesempio è $r_1(x)w_1(x)r_2(x)w_2(x)r_3(y)w_1(y)$, il quale è CSR ma viola il 2PL poiché t_1 rilascia il lock su x prima di acquisire quello su y .

6.4 Fallimenti e Locking a Due Fasi Stretto (S2PL)

Rimuovendo l'ipotesi di "commit-proiezione", si deve considerare il rischio che le transazioni falliscano, introducendo la possibilità di **letture sporche**. Se t_i legge un dato modificato da t_k e t_k abortisce dopo che t_i ha già comunicato all'esterno o effettuato il commit, si verifica un'incoerenza insanabile. Per evitare ciò, si adotta il **Locking a due fasi stretto (S2PL)**, che impone una condizione aggiuntiva: **tutti i lock acquisiti da una transazione devono essere mantenuti fino al momento del commit o dell'abort**.

Nella gestione pratica dei lock, ogni richiesta è associata a un **timeout**. Se un lock non può essere concesso immediatamente, la transazione viene posta in attesa. Se la risorsa non si libera entro il tempo massimo stabilito, la transazione viene abortita e, eventualmente, rilanciata dall'applicazione.

6.5 Granularità e Stallo

Il lock può essere applicato a diversi livelli di **granularità**: dall'intera base di dati alle singole ennuple, passando per tabelle e blocchi. Una granularità grossa blocca molte risorse inutilmente, mentre una fine richiede un elevato overhead di gestione. I sistemi implementano spesso l'**escalation** dei lock: se una

transazione accumula troppi lock su singole ennuple di una tabella, il sistema converte tali lock in un unico lock sull'intera tabella.

L'uso dei lock introduce il rischio di **stallo (deadlock)**, ovvero una situazione di attese incrociate dove due o più transazioni attendono risorse detenute l'una dall'altra (ad esempio t_1 ha $r_lock(x)$ e aspetta $w_lock(y)$, mentre t_2 ha $r_lock(y)$ e aspetta $w_lock(x)$). Lo stallo corrisponde a un ciclo nel **grafo delle attese**. Le tecniche di gestione includono:

1. **Timeout**: aborto della transazione dopo un tempo di attesa eccessivo.
2. **Rilevamento**: ricerca periodica di cicli nel grafo delle attese.
3. **Prevenzione**: interruzione preventiva di transazioni giudicate “sospette”.

6.6 Livelli di Isolamento e Inserimenti Fantasma

Lo standard SQL:1999 (e JDBC) definisce livelli di isolamento con implementazioni specifiche:

- **Read Uncommitted**: non utilizza lock in lettura, permettendo letture sporche e inconsistenti.
- **Read Committed**: utilizza lock in lettura ma senza il vincolo 2PL, evitando letture sporche ma non quelle inconsistenti.
- **Repeatable Read**: implementa il protocollo S2PL sui dati letti, evitando letture inconsistenti e aggiornamenti fantasma, ma non gli inserimenti fantasma.
- **Serializable**: il livello massimo che evita ogni anomalia.

L'**inserimento fantasma (phantom)** si verifica quando una transazione esegue una ricerca basata su un predicato (es. `SELECT ... WHERE volo=X`) e una transazione concorrente inserisce nuove ennuple che soddisfano tale condizione. Per prevenirlo, il livello serializable utilizza i **lock di predicato**, che impediscono l'accesso in scrittura a tutti i dati, attuali o futuri, che soddisfano il predicato di lettura. Questi vengono implementati bloccando l'intera relazione o, più efficientemente, bloccando le porzioni corrispondenti dell'indice sugli attributi della condizione.

7 Controllo di concorrenza basato su timestamp

Il controllo della concorrenza basato su timestamp rappresenta una tecnica fondamentale e alternativa al protocollo di locking a due fasi (*2PL*). Sebbene nella pratica commerciale si utilizzino spesso varianti più sofisticate, l'intuizione teorica rimane il pilastro del coordinamento ottimistico delle transazioni. Un **timestamp** è un identificatore univoco che definisce un ordinamento totale su tutti gli eventi di un sistema. Nel contesto dei DBMS, ogni transazione riceve un timestamp che rappresenta esattamente l'istante del suo inizio.

La logica sottostante stabilisce che uno schedule venga accettato dal protocollo (definito come *schedule TS*) solo se l'ordine dei conflitti tra le operazioni riflette fedelmente l'ordinamento seriale delle transazioni indotto dai loro timestamp. Di conseguenza, ogni schedule che soddisfa i criteri del timestamp è intrinsecamente un membro della classe degli schedule conflict-serializzabili (*CSR*). Metaforicamente, il timestamp impedisce i paradossi temporali nella base di dati: una transazione non può leggere o scrivere dati che appartengono al suo “futuro” relativo, né può modificare un passato che è già stato osservato da transazioni più giovani.

7.1 Meccanismi di controllo e variabili di stato

Per implementare operativamente questo protocollo, il sistema deve mantenere per ogni oggetto x presente nella base di dati due variabili di stato specifiche:

- $RTM(x)$: indica il timestamp della transazione più “giovane” (ovvero con il timestamp più alto) che ha effettuato una lettura dell'oggetto x .
- $WTM(x)$: indica il timestamp della transazione più “giovane” che ha effettuato una scrittura dell'oggetto x .

Quando lo scheduler riceve una richiesta di operazione da una transazione dotata di timestamp t , applica le seguenti regole decisionali:

7.1.1 Operazione di lettura $r_t(x)$

Il sistema verifica se la transazione sta tentando di leggere un valore che è già stato sovrascritto da una transazione che dovrebbe venire “dopo” di lei nel tempo logico.

- Se $t < WTM(x)$, la richiesta viene respinta. La transazione viene immediatamente uccisa e deve essere rilanciata con un nuovo timestamp.
- In tutti gli altri casi, la richiesta è accolta. Il sistema aggiorna il valore di $RTM(x)$ ponendolo pari al valore maggiore tra quello attuale e t ($RTM(x) = \max(RTM(x), t)$).

7.1.2 Operazione di scrittura $w_t(x)$

Il sistema verifica se la scrittura è compatibile con le letture e le scritture già avvenute.

- Se $t < WTM(x)$ oppure $t < RTM(x)$, la richiesta è respinta e la transazione viene uccisa. Questo accade perché un'altra transazione più giovane ha già scritto l'oggetto o, peggio, lo ha già letto, rendendo la scrittura di t un'interferenza con il passato di una transazione successiva.
- Altrimenti, la richiesta viene accolta e il valore di $WTM(x)$ viene aggiornato a t .

Per estendere il funzionamento del protocollo anche in scenari dove non si assume l'ipotesi di commit-proiezione (ovvero dove le transazioni possono fallire), è necessario proteggere le scritture fino al momento del commit, solitamente integrando meccanismi di *2PL* esclusivamente sulle operazioni di scrittura.

7.1.3 La variante “Thomas Write Rule”

Esiste una piccola ma significativa variante ottimizzativa per le regole di scrittura. Se una scrittura $w_t(x)$ arriva in ritardo rispetto a un'altra scrittura ($t < WTM(x)$), ma nessuno ha ancora letto l'oggetto ($t \geq RTM(x)$), il sistema può semplicemente decidere di ignorare la richiesta invece di uccidere la transazione. Poiché nessuno ha avuto necessità di leggere quel dato specifico prodotto da t , la sua omissione non altera la coerenza logica dello schedule. La regola modificata prevede quindi:

- Se $t < RTM(x)$, la transazione viene uccisa.
- Se $t < WTM(x)$, la richiesta viene ignorata.
- Altrimenti, la richiesta è accolta e $WTM(x) = t$.

7.2 Confronto tra 2PL e Timestamp

I protocolli *2PL* e *TS* sono classi di schedule incomparabili: esistono schedule accettati dal timestamp ma non dal *2PL*, e viceversa.

- **Schedule in TS ma non in 2PL:** $r_1(x)w_1(x)r_2(x)w_2(x)r_0(y)w_1(y)$.
- **Schedule in 2PL ma non in TS:** $r_2(x)w_2(x)r_1(x)w_1(x)r_1(y)r_2(x)w_2(x)r_1(x)w_1(x)$ (assumendo che i pedici riflettano i timestamp).

Fondamentalmente, il **2PL** è “pessimista”: esso preferisce porre le transazioni in attesa tramite i lock per prevenire i conflitti. Al contrario, il **TS** è “ottimista”: permette alle transazioni di procedere e interviene solo ex-post uccidendo e rilanciando quelle che violano l'ordine cronologico. Sebbene il *2PL* possa causare deadlock (situazione meno frequente nel *TS* per l'assenza di lock in lettura), la scelta tra i due dipende dal costo applicativo delle ripartenze rispetto a quello delle attese. Storicamente, il *2PL* è stato il protocollo dominante, ma i sistemi moderni (come PostgreSQL) virano verso l'integrazione di timestamp e multiversioni.

7.3 Controllo di concorrenza multiversione (MVCC)

Il controllo multiversione nasce dall'osservazione che uccidere una transazione solo perché tenta di leggere un dato “vecchio” è spesso inefficiente. L'idea di base è che **ogni operazione di scrittura generi una nuova copia (versione) dell'oggetto associata al proprio WTM** . In questo modo, un oggetto x non ha un unico valore, ma una serie di versioni con timestamp di scrittura diversi.

7.3.1 Meccanismo di lettura e scrittura MVCC

- **Lettura $r_t(x)$:** è sempre accettata. Il sistema seleziona la versione dell'oggetto x_k più adatta alla transazione. Se t è maggiore di tutti i timestamp delle copie disponibili, si sceglie l'ultima versione. Altrimenti, si seleziona la versione k tale che $WTM_k(x) < t < WTM_{k+1}(x)$.
- **Scrittura $w_t(x)$:** se $t < RTM(x)$, la richiesta è rifiutata poiché una transazione più giovane ha già letto un valore che logicamente avrebbe dovuto essere prodotto dopo la scrittura di t . In caso contrario, viene generata una nuova versione di x con $WTM(x) = t$.

Le versioni obsolete vengono rimosse fisicamente dal sistema (operazione di *garbage collection*) solo quando è certo che nessuna transazione attiva sia più interessata a leggerle.

7.4 Implementazioni nei DBMS reali: DB2 e PostgreSQL

I DBMS commerciali adottano approcci ibridi:

- **DB2:** implementa sostanzialmente il *2PL* stretto per garantire l'isolamento.
- **Postgres:** utilizza un approccio multiversione (*MVCC*), con l'aggiunta di lock sulle scritture.

In Postgres, il comportamento varia in base al livello di isolamento impostato:

1. **Read Committed (Default):** le letture operano sui dati andati in commit al momento dell'inizio della singola istruzione **SELECT**.
 1. Le scritture utilizzano il *2PL* stretto, acquisendo e mantenendo i lock fino al commit.
2. **Repeatable Read:** le letture multiversione operano sui dati in commit all'inizio della transazione (identificato dalla prima operazione di lettura/scrittura, non dal **BEGIN**).
 1. Per le scritture, vige il *2PL* stretto con una condizione di "rispetto delle versioni": una transazione T non può modificare dati che siano stati alterati da un'altra transazione T' dopo l'avvio di T .
3. **Serializable:** simile al livello precedente, ma aggiunge la verifica di cicli di conflitti lettura/scrittura (dipendenze r/w).
 1. Questa funzionalità, introdotta dalla versione 9.1, è implementata con una variante di lock di predicato che traccia l'ordine delle operazioni senza bloccare, verificando l'eventuale presenza di cicli solo al momento del commit.

7.5 Analisi delle anomalie e sperimentazione pratica

L'efficacia di questi protocolli può essere testata simulando anomalie note.

- **Perdita di aggiornamento (Lost Update):** Se due transazioni tentano di aggiornare lo stesso saldo leggendolo contemporaneamente in locale e riscrivendolo, il comportamento dipende dal sistema. In Postgres, ai livelli **Serializable** e **Repeatable Read**, la seconda transazione viene abortita perché il dato è stato modificato dopo il suo inizio. Nel *2PL* puro, la situazione porterebbe a uno stallo (deadlock) gestito dal lock manager.
- **Lettura inconsistente:** In scenari dove una transazione effettua più letture intervallate da una scrittura esterna, i livelli multiversione garantiscono che la transazione veda sempre la medesima "fotografia" dei dati scattata al suo inizio.

Il controllo di concorrenza "ignora la semantica delle operazioni": esso non è in grado di comprendere se una lettura influenzi logicamente una scrittura successiva (dipendenza funzionale applicativa). Pertanto, in scenari critici dove i valori letti determinano i valori da scrivere, l'uso del livello di isolamento massimo è imperativo per evitare che il sistema, agendo in modo troppo ottimista, permetta l'avvio di transazioni che non potrà portare a termine coerentemente.

8 Gestione delle transazioni nelle basi di dati distribuite

Una base di dati distribuita si configura come un sistema in cui sia i dati che il controllo non risiedono su un unico elaboratore, ma sono ripartiti su server multipli denominati nodi. Questi nodi operano in modo coordinato tra loro per offrire una visione unitaria del sistema. La filosofia architetturale prevede che le operazioni siano, per quanto possibile, delegate ai singoli nodi competenti per la porzione di dato interessata, minimizzando lo scambio di informazioni non necessario.

Uno schema logico di tale sistema mostra un dominio D che racchiude diversi cilindri di memorizzazione, ognuno composto da un modulo software DBMS e da una componente fisica DATI. All'interno del dominio D , i nodi sono interconnessi da archi bidirezionali che rappresentano i canali di comunicazione e coordinamento. Una transazione T entra nel sistema dall'esterno e viene presa in carico dall'infrastruttura distribuita che ne gestisce l'instradamento e l'esecuzione.

8.1 Architetture distribuite: Partizionamento e Replicazione

L'organizzazione fisica dei dati in un ambiente distribuito segue due strategie principali, spesso combinate tra loro per massimizzare prestazioni e disponibilità:

Il partizionamento, noto anche come sharding, consiste nella suddivisione logica e fisica dei dati tra i vari nodi del sistema. Ogni nodo (Machine) ospita una porzione specifica del database (Chunk). Ad esempio, si può immaginare un'architettura con sei macchine collegate a un bus di comunicazione centrale: la Machine 1 ospita i Chunk 1 e 2, la Machine 2 i Chunk 3 e 4, e la Machine 3 i Chunk 5 e 6. In questo modo, nessuna macchina possiede l'intero dataset, distribuendo il carico di memorizzazione e computazione.

La replicazione prevede invece che gli stessi dati siano memorizzati su più nodi contemporaneamente. Riprendendo l'esempio precedente, la Machine 4 potrebbe ospitare nuovamente i Chunk 1 e 2, la Machine 5 i Chunk 3 e 4, e la Machine 6 i Chunk 5 e 6. Questa ridondanza assicura che, in caso di guasto di una macchina (ad esempio la Machine 1), i dati siano ancora accessibili attraverso la sua replica (Machine 4), aumentando la tolleranza ai guasti del sistema.

8.2 Tipologie di transazioni distribuite

Nello scenario dei sistemi distribuiti, è fondamentale distinguere tra diverse modalità di interazione con i dati, a seconda del grado di trasparenza e coinvolgimento dei nodi:

Una transazione remota si verifica quando un utente, operando in un contesto locale, interagisce con una base di dati esterna. Un esempio SQL tipico è l'aggiornamento di un ufficio situato su un server remoto:

```
1 UPDATE scott.dept@sales.us.americas.acme_auto.com
2 SET loc = 'NEW YORK'
3 WHERE deptno = 10;
4 UPDATE scott.emp@sales.us.americas.acme_auto.com
5 SET deptno = 11
6 WHERE deptno = 10;
7 COMMIT;
```

In questo caso, l'utente specifica esplicitamente l'indirizzo del nodo remoto per ogni operazione.

Una transazione distribuita propriamente detta interagisce con più basi di dati contemporaneamente all'interno della medesima unità logica di lavoro. L'utente può aggiornare una tabella su un nodo remoto e una tabella sul nodo locale o su un altro nodo remoto nello stesso blocco transazionale:

```
1 UPDATE scott.dept@sales.us.americas.acme_auto.com
2 SET loc = 'NEW YORK'
3 WHERE deptno = 10;
4 UPDATE scott.emp
5 SET deptno = 11
6 WHERE deptno = 10;
7 COMMIT;
```

Spesso, la natura distribuita della transazione è resa trasparente all'utilizzatore attraverso l'uso di alias o viste globali. Il DBMS si occupa di inoltrare le operazioni ai nodi competenti senza che l'utente debba conoscere la collocazione fisica dei dati, come nel comando semplificato:

```

1 UPDATE scott.dept SET loc = 'NEW YORK' WHERE deptno = 10;
2 UPDATE scott.emp SET deptno = 11 WHERE deptno = 10;
3 COMMIT;

```

8.3 Consistenza e Durabilità nei sistemi distribuiti

In contesti caratterizzati esclusivamente dal partizionamento (sharding), **la distribuzione dei dati non influenza direttamente le proprietà di consistenza e durabilità**, che rimangono ancorate ai meccanismi locali dei singoli nodi.

La consistenza non dipende dalla distribuzione poiché i vincoli di integrità (come chiavi primarie o esterne) sono gestiti localmente. La tecnologia DBMS attuale, infatti, raramente gestisce vincoli di integrità distribuiti che coinvolgono tabelle residenti su nodi diversi, delegando la coerenza al livello applicativo o a controlli locali.

La durabilità è garantita in modo indipendente da ogni nodo attraverso i meccanismi di recupero tradizionali. Ogni sistema mantiene il proprio log delle operazioni, esegue checkpoint periodici e produce dump della base di dati locale. In caso di guasto di un nodo, il ripristino dei dati avviene utilizzando esclusivamente queste risorse locali, indipendentemente dallo stato degli altri nodi della rete.

Gli aspetti critici introdotti dalla distribuzione riguardano invece l'isolamento (gestito tramite il controllo della concorrenza) e l'atomicità (gestita tramite il protocollo di commit atomico).

8.4 Controllo della concorrenza globale

La teoria della concorrenza stabilisce che **la semplice serializzabilità locale su ogni singolo nodo non è una condizione sufficiente a garantire la serializzabilità globale dell'intero sistema**. Si considerino due transazioni, t_1 e t_2 , operanti su due nodi A e B:

- t_1 : $r_{1A}(x)$ $w_{1A}(x)$ $r_{1B}(y)$ $w_{1B}(y)$
- t_2 : $r_{2B}(y)$ $w_{2B}(y)$ $r_{2A}(x)$ $w_{2A}(x)$

Sui singoli nodi potremmo avere le seguenti sequenze locali serializzabili:

- Nodo A: $r_{1A}(x)$ $w_{1A}(x)$ $r_{2A}(x)$ $w_{2A}(x)$ (ordine $t_1 \rightarrow t_2$)
- Nodo B: $r_{2B}(y)$ $w_{2B}(y)$ $r_{1B}(y)$ $w_{1B}(y)$ (ordine $t_2 \rightarrow t_1$)

Sebbene ogni nodo sia coerente internamente, l'ordine globale è contraddittorio (un ciclo nel grafo dei conflitti globale), violando la serializzabilità. Nella pratica, tuttavia, se i lock in scrittura vengono mantenuti fino al momento del commit globale (che coinvolge tutti i nodi contemporaneamente), la serializzabilità globale viene garantita automaticamente dall'ordinamento temporale dei commit e dalla corretta lettura delle versioni dei dati.

8.5 Tipologie di guasto nei sistemi distribuiti

Un sistema distribuito deve far fronte a una gamma di malfunzionamenti più ampia rispetto a un sistema centralizzato. Oltre ai guasti locali sui singoli nodi (che seguono le dinamiche tradizionali di crash), si aggiungono le problematiche legate alla rete.

La perdita di messaggi è una criticità rilevante che può lasciare i protocolli di coordinamento in uno stato di incertezza. Per mitigare il problema, ogni messaggio importante è seguito da una conferma di ricezione (ack). Tuttavia, il sistema deve gestire il caso in cui vadano perduti sia i messaggi originali che le relative conferme.

Il partizionamento della rete si verifica quando la connettività tra i nodi si interrompe in modo tale da dividere il sistema in due o più sottogruppi isolati. In questa situazione, i nodi di una partizione non possono comunicare con quelli dell'altra, rendendo impossibile raggiungere un consenso globale senza protocolli specifici.

8.6 Il protocollo di Commit a Due Fasi (2PC)

L'obiettivo del protocollo di commit a due fasi è assicurare **latomicità distribuita**, ovvero garantire che tutte le parti coinvolte in una transazione prendano la medesima decisione: o tutte effettuano il commit o tutte effettuano l'abort. Il protocollo funziona analogamente a un contratto legale mediato da un notaio. I partecipanti alla transazione sono definiti Resource Manager (RM), mentre il coordinatore dell'operazione è chiamato Transaction Manager (TM). Il coordinatore può essere uno dei partecipanti stessi.

Il protocollo si sviluppa in due fasi distinte, preparazione e conferma, separate da un momento decisionale centrale:

1. Fase di preparazione: Il coordinatore interroga i partecipanti per sapere se sono pronti e disponibili a rendere persistenti le modifiche.
2. Decisione: Il coordinatore raccoglie i pareri. Se tutti sono favorevoli, la decisione è "global commit"; se anche un solo partecipante è contrario o non risponde, la decisione è "global abort".
3. Fase di conferma: Il coordinatore comunica la decisione a tutti i partecipanti, i quali devono confermare l'avvenuta ricezione ed esecuzione.

Tutto il processo si basa sullo scambio rapido di messaggi, sulla scrittura di record specifici nei log locali per garantire la persistenza della decisione e sull'invio di conferme (ack) per chiudere il protocollo.

8.6.1 Record nel Log del Coordinatore (TM)

Il coordinatore deve registrare le fasi del protocollo per poter gestire eventuali riavvii post-crash:

- *prepare*: Questo record contiene l'identificativo della transazione e l'elenco di tutti i partecipanti coinvolti. Serve al coordinatore per ricordare a chi ha chiesto la disponibilità al commit.
- *global commit* o *global abort*: Rappresenta la decisione definitiva presa dal coordinatore dopo aver consultato i partecipanti. Una volta scritto questo record, la decisione è considerata ufficiale.
- *complete*: Indica la conclusione del protocollo, ovvero che tutti i partecipanti hanno confermato di aver ricevuto ed eseguito la decisione globale.

8.6.2 Record nel Log dei Partecipanti (RM)

I partecipanti utilizzano i record standard di gestione transazionale (*begin*, *insert*, *delete*, *update*, *commit*, *abort*) integrati da un record specifico del protocollo distribuito:

- *ready*: Questo record conferma che il partecipante ha verificato internamente di poter completare la transazione (le risorse sono disponibili, i vincoli di integrità sono soddisfatti, ecc.). Scrivere *ready* nel log è un atto irrevocabile: da questo momento il partecipante perde la propria autonomia decisionale e si impegna a eseguire qualunque decisione (commit o abort) verrà presa dal coordinatore. Il record contiene i riferimenti alla transazione e al coordinatore.

8.7 Prima fase del protocollo: Preparazione

L'iter inizia quando il coordinatore scrive il record di *prepare* nel proprio log e invia simultaneamente un messaggio di *prepare* a tutti i partecipanti, fissando un tempo massimo di attesa (timeout) per le risposte.

Ogni singolo partecipante che riceve il messaggio di *prepare* esegue un controllo interno. Se è in grado di procedere, scrive il record di *ready* nel proprio log e trasmette al coordinatore un messaggio di *ready*. In caso contrario, ovvero se non può garantire il commit, può inviare un messaggio di *not-ready* oppure non inviare nulla, lasciando scadere il timeout del coordinatore.

Il coordinatore rimane in ascolto delle risposte fino alla scadenza del timeout. Se riceve messaggi di *ready* da tutti i partecipanti, scrive il record di *global commit* nel proprio log. Se manca anche una sola risposta o se riceve un messaggio negativo, scrive un record di *global abort*.

8.8 Seconda fase del protocollo: Conferma

Una volta presa la decisione globale, il coordinatore la trasmette a tutti i partecipanti che avevano risposto *ready* e a quelli da cui non era giunta risposta, fissando un secondo timeout per la ricezione degli *ack*.

Ogni partecipante, alla ricezione della decisione globale, scrive il record corrispondente (*commit* o *abort*) nel proprio log locale e invia un messaggio di *ack* al coordinatore. Contestualmente, implementa fisicamente la decisione con le opportune scritture sui dati e il rilascio dei lock.

Il coordinatore attende i messaggi di *ack*. Se allo scadere del timeout manca qualche conferma da parte dei partecipanti che erano in stato di *ready*, il coordinatore provvede a reinviare la decisione a quei nodi specifici. Solo quando tutti i partecipanti hanno confermato la ricezione, il coordinatore scrive il record di *complete* nel log, chiudendo definitivamente la transazione distribuita.

8.9 Diagramma temporale del Commit a due fasi

Il flusso del protocollo 2PC può essere visualizzato su due assi temporali paralleli: uno per il Transaction Manager (TM) e uno per il Resource Manager (RM).

1. Il TM invia un messaggio di *prepare* (linea tratteggiata discendente verso RM).
2. L'RM, ricevuto il messaggio, scrive il record di *Ready* (freccia verso l'alto sull'asse RM) e invia il messaggio di *ready* (linea tratteggiata ascendente verso TM).
3. Il TM riceve i messaggi, prende la *Global Decision* (freccia verso l'alto sull'asse TM) e invia il messaggio di *decision* (linea discendente).
4. L'RM riceve la decisione, esegue la *Local Decision* (freccia verso l'alto sull'asse RM) e invia l' *ack* (linea ascendente).
5. Il TM riceve gli *ack* e scrive il record di *Complete* (freccia verso l'alto sull'asse TM).

In questo schema, si definisce finestra di incertezza l'intervallo temporale che intercorre tra il momento in cui l'RM scrive il record di *ready* e il momento in cui riceve il messaggio di decisione globale dal TM.

8.10 Incertezza e procedure di ripristino

Lo stato di *ready* è critico per un partecipante poiché comporta la perdita totale della propria autonomia decisionale. Durante la finestra di incertezza, il partecipante è obbligato ad attendere la decisione del coordinatore. Se il coordinatore fallisce in questo lasso di tempo, il partecipante rimane bloccato in uno stato incerto; poiché i lock sulle risorse sono solitamente mantenuti fino al *commit* o all'*abort*, i dati coinvolti rimangono inaccessibili ad altre transazioni, causando potenziali colli di bottiglia.

8.10.1 Recovery del partecipante

In fase di ripristino post-crash, un partecipante analizza l'ultimo record nel log per ogni transazione attiva:

- Se l'ultimo record è un *commit*, si esegue il *redo* delle azioni per garantirne la durabilità.
- Se l'ultimo record è un'azione generica o un *abort*, la transazione deve essere annullata tramite l'operazione di *undo*.
- Se l'ultimo record è un *ready*, il partecipante è in uno stato di incertezza. Non potendo decidere autonomamente, deve contattare il coordinatore (o attendere che sia il coordinatore a ricontattarlo) per conoscere l'esito della transazione.

8.10.2 Recovery del coordinatore

Il coordinatore segue una logica speculare analizzando il proprio log:

- Se l'ultimo record è *complete*, non è necessaria alcuna azione poiché il protocollo si era concluso correttamente.
- Se l'ultimo record è una decisione (*global commit* o *abort*), la scelta è definitiva e immutabile. Tuttavia, non essendoci il *complete*, non è certo che tutti i partecipanti siano stati informati, quindi il coordinatore deve ripetere la seconda fase del protocollo inviando nuovamente la decisione.

- Se l'ultimo record è un *prepare*, potrebbero esserci partecipanti in attesa della decisione. La soluzione più conservativa e semplice è decidere per un *global abort*. In alternativa, il coordinatore potrebbe tentare di ripetere la prima fase interrogando nuovamente i partecipanti.

8.11 Gestione dei guasti e dei messaggi perduti

Il protocollo 2PC gestisce in modo uniforme crash, perdite di messaggi e partizionamenti della rete applicando la regola del fallimento della fase in assenza di conferme.

Se la mancanza di risposta avviene durante la prima fase (preparazione), il coordinatore solitamente decide di abortire la transazione distribuita per precauzione. Se la mancanza di risposta (mancato ack) avviene durante la seconda fase (conferma), il coordinatore è obbligato a ripetere la trasmissione della decisione finché non riceve conferma, poiché la decisione è già stata presa e registrata nel log tra la prima e la seconda fase, e non può più essere modificata.

9 Consistenza nelle basi di dati distribuite

I moderni sistemi di gestione di basi di dati (DBMS) devono affrontare la necessità di scalare per supportare i requisiti prestazionali e di carico delle applicazioni contemporanee. Esistono due direttrici principali per l'espansione: la scalabilità verticale (scale up) e la scalabilità orizzontale (scale out). La scalabilità verticale prevede il potenziamento del singolo server attraverso l'allocazione di risorse hardware superiori, quali CPU più potenti, una maggiore quantità di memoria primaria e dischi di capacità elevata; tuttavia, questa strategia incontra un limite fisico insormontabile nella quantità massima di risorse che possono essere concentrate in una singola macchina. La scalabilità orizzontale prevede invece l'aumento del numero di server fisici impiegati, utilizzando tecniche di partizionamento (sharding) e replicazione. In questo scenario, il limite principale è rappresentato dal sovraccarico di comunicazione (communication overhead) tra i nodi, ma non è l'unico vincolo.

9.1 Partizionamento e Replicazione

L'architettura distribuita si fonda su due concetti chiave che possono essere combinati tra loro:

- **Partizionamento:** consiste nella suddivisione logica e fisica dei dati tra i vari nodi del sistema.
- **Replicazione:** prevede la memorizzazione degli stessi dati su più nodi differenti.

Uno schema esemplificativo vede una configurazione con sei macchine interconnesse. In un sistema puramente partizionato, la Machine 1 ospita i Chunk 1 e 2, la Machine 2 ospita i Chunk 3 e 4, e la Machine 3 ospita i Chunk 5 e 6. Quando si introduce la replicazione, i medesimi dati vengono duplicati su altre macchine: la Machine 4 replica il contenuto della Machine 1 (Chunk 1 e 2), la Machine 5 replica la Machine 2 (Chunk 3 e 4), e la Machine 6 replica la Machine 3 (Chunk 5 e 6).

Mantenere la consistenza tra le repliche di una base di dati distribuita rappresenta una sfida complessa. Si consideri un esempio con due repliche del database dove il saldo iniziale è $Bal = 1000$. Vengono generati due eventi concorrenti: l'Evento 1 aggiunge 1000 € e l'Evento 2 aggiunge un interesse del 5%. Se sulla prima replica viene eseguito l'ordine (1, 2), il saldo finale sarà $(1000 + 1000) \times 1.05 = 2100$. Se sulla seconda replica viene eseguito l'ordine (2, 1), il saldo finale sarà $(1000 \times 1.05) + 1000 = 2050$. Questa divergenza evidenzia la difficoltà nel garantire la coerenza tra le copie.

9.2 Sistemi distribuiti: il trade-off tra Safety e Liveness

In un'elaborazione distribuita operante in un contesto non affidabile (unreliable), è impossibile garantire simultaneamente le proprietà di safety e liveness.

- **Safety:** descrive il principio per cui “non succede niente di male”. La consistenza, intesa come coerenza tra le copie, è una proprietà di safety: tutte le risposte fornite ai client devono essere corrette secondo una determinata nozione di correttezza.
- **Liveness:** descrive il principio per cui “prima o poi succede qualcosa di buono”. La disponibilità (availability) è una proprietà di liveness: ogni richiesta effettuata da un client deve ricevere, prima o poi, una risposta.
- **Unreliability:** indica la presenza di fallimenti intrinseci, quali partizionamenti della rete (network partitioning), crash dei nodi (crash failures) o comportamenti malevoli dei nodi (Byzantine failures).

9.3 Il Teorema CAP

Il teorema CAP (introdotto da Eric Brewer) formalizza il trade-off tra tre proprietà in un sistema distribuito composto da un insieme di nodi $G_1, \dots, G_n$ distribuiti geograficamente, dove i client eseguono operazioni di lettura e scrittura:

1. **Consistenza (Atomic consistency)**: definita dalla semantica complessiva del servizio. Le sequenze di operazioni devono produrre lo stesso effetto che avrebbero se fossero eseguite su un singolo server centralizzato. Il client deve percepire il servizio come atomico e le risposte devono essere coerenti.
2. **Disponibilità (Availability)**: ogni richiesta riceve una risposta (eventual response). Una risposta eccessivamente lenta può essere considerata errata o equivalente a un fault.
3. **Tolleranza al partizionamento (Network partitioning)**: descrive la suddivisione della rete in gruppi che non possono comunicare tra loro. Poiché i messaggi possono essere ritardati o persi, l'inaffidabilità della comunicazione è un dato di fatto del sistema.

L'enunciato del teorema CAP stabilisce che in un network soggetto a partizionamento, è impossibile implementare una memoria condivisa read/write atomica che fornisca risposta a ogni richiesta. La dimostrazione intuitiva prevede che se due nodi sono separati da un partizionamento, un aggiornamento su uno non può essere propagato all'altro. Di conseguenza, l'altro nodo potrà o non rispondere (violando la disponibilità) o rispondere con un dato obsoleto (violando la consistenza).

9.3.1 Implicazioni pratiche e compromessi

Nella costruzione di sistemi reali si adottano diversi compromessi:

- **Best effort availability**: se la consistenza è un vincolo inderogabile, la disponibilità passa in secondo piano. Un esempio è il Google Lock Service (Chubby), che supporta GFS e Big Table. Fornisce consistenza forte tramite un design primary-backup; se i server sono partizionati, il servizio diventa non disponibile.
- **Best effort consistency**: se la disponibilità e la velocità di risposta sono prioritarie, si tollerano inconsistenze temporanee. Esempi includono il caching di contenuti web (Akamai) e i sistemi basati su eventual consistency.

9.3.2 Consistenza basata su Quorum

Un meccanismo per bilanciare consistenza e disponibilità è l'uso dei quorum:

- N_W : numero minimo di nodi che devono confermare una scrittura.
- N_R : numero minimo di nodi che devono confermare una lettura.
- La condizione $N_R + N_W > n$ garantisce letture fortemente consistenti (strongly consistent reads), poiché l'intersezione tra l'insieme di lettura e quello di scrittura non è mai vuota.
- La condizione $N_W > n/2$ serve a evitare conflitti scrittura-scrittura, garantendo l'isolamento.

I trade-off possono essere definiti dinamicamente:

- **Soglie di “out-of-date”-ness**: si tollerano inconsistenze temporanee per limitare la non disponibilità solo a partizionamenti molto lunghi.
- **Stato dell'esecuzione**: in una prenotazione aerea, si può tollerare l'inconsistenza quando ci sono molti posti disponibili, ma la consistenza deve diventare stretta quando i posti rimasti sono pochi.
- **Caratteristiche dei dati**: l'inventario di un e-commerce può essere disallineato, ma il carrello degli acquisti deve essere coerente.
- **Tipo di operazione**: le letture possono tollerare inconsistenze (es. navigazione catalogo), mentre le scritture no (es. acquisto prodotto).
- **Utente o Gerarchia**: gli utenti possono essere partizionati geograficamente con consistenza maggiore entro la propria partizione. Scendendo in un partizionamento gerarchico, si possono fornire livelli di consistenza più elevati.

9.4 Il problema del Consenso

Il consenso riguarda la capacità di un insieme di nodi $G_1, \dots, G_n$, ciascuno con un valore iniziale v_i , di accordarsi su un unico valore di output. Le proprietà richieste sono:

- **Agreement:** tutti i nodi devono restituire lo stesso valore (safety).
- **Validity:** il valore di output deve essere stato proposto come input da almeno un nodo (safety).
- **Termination:** ogni nodo deve prima o poi restituire un valore (liveness).

Gli algoritmi di consenso assicurano la safety ritornando sempre risultati corretti e garantiscono la liveness se la maggioranza dei nodi è attiva, anche in presenza di ritardi, partizionamenti o riordinamento di messaggi.

9.4.1 Macchine a Stati Replicate (Replicated State Machine)

Gli algoritmi di consenso si basano sul concetto di macchina a stati replicata. Ogni nodo possiede una copia identica di una macchina a stati e opera su un log replicato (replicated log), ovvero una sequenza di istruzioni. Poiché le macchine sono deterministiche, se applicano gli stessi comandi nello stesso ordine dal medesimo log, raggiungeranno lo stesso stato. L'algoritmo di consenso ha la responsabilità di mantenere allineati i log tra i nodi attraverso un modulo di consenso che riceve i comandi dai client e comunica con gli altri nodi per far convergere i log.

9.5 L'algoritmo RAFT

RAFT è un algoritmo di consenso introdotto da Leslie Lamport nel 1989 (come versione semplificata e didattica di Paxos) per essere più comprensibile e adatto a sistemi reali. Il problema del consenso in RAFT viene decomposto in tre sottoproblemi:

1. **Elezione del leader:** scelta di un nuovo leader al fallimento del precedente.
2. **Replicazione del log:** il leader accetta le entry dai client e le replica nel cluster.
3. **Safety:** garanzia che se un nodo applica un'operazione, nessun altro nodo applicherà un comando diverso per la stessa posizione.

9.5.1 Stati e tempo in RAFT

Un cluster RAFT solitamente contiene 5 server per tollerare 2 fallimenti simultanei. Ogni server può trovarsi in uno di tre stati:

- **Leader:** gestisce le richieste dei client. Se un client contatta un follower, viene reindirizzato al leader.
- **Follower:** stato passivo; risponde alle richieste dei leader o dei candidate.
- **Candidate:** stato temporaneo utilizzato durante le elezioni.

Il tempo è suddiviso in “terms” di lunghezza arbitraria, ognuno dei quali inizia con un'elezione. I term agiscono come un orologio logico (logical clock) per individuare informazioni obsolete. Se un server scopre di avere un progressivo di term inferiore a un altro, si aggiorna; se un leader o un candidate scopre un term superiore, torna immediatamente allo stato di follower. Le richieste riferite a term superati vengono rigettate.

9.5.2 Comunicazione e Invarianti

La comunicazione avviene tramite due tipi di RPC (Remote Procedure Call):

- **RequestVOTE:** avviata dai candidati durante le elezioni.
- **AppendEntries:** avviata dai leader per replicare le entry del log e fungere da heartbeat.

RAFT si fonda su rigide invarianti per mantenere l'allineamento:

- **Election safety:** al massimo un leader per ogni term.
- **Leader append-only:** il leader non sovrascrive né cancella mai le proprie entry.
- **Log matching:** entry con stesso indice e stesso term implicano comandi identici e log identici nelle posizioni precedenti.
- **Leader completeness:** le entry committate in un term sono presenti nei leader di tutti i term successivi.
- **State machine safety:** nessuna macchina a stati applicherà mai comandi diversi per lo stesso indice.

9.5.3 Processo di Elezione e Replicazione

Un server inizia come follower. Se non riceve heartbeat entro un “election timeout”, diventa candidate, incrementa il term, vota per se stesso e invia RequestVote. Vince chi ottiene la maggioranza dei voti nel cluster. Per evitare split vote infiniti (dove nessuno ottiene la maggioranza), si usa un “randomized election timeout”.

Una volta eletto, il leader riceve comandi dai client e li aggiunge al proprio log. Invia AppendEntries in parallelo. Una entry è considerata “committed” quando è replicata sulla maggioranza dei server. Il commit di una entry implica il commit automatico di tutte le entry precedenti nel log. Il leader include l'indice committato più alto in ogni AppendEntries per informare i follower.

In caso di inconsistenze (disallineamento dei log dovuto a crash), il leader forza i follower a uniformarsi al suo log. Individua l'ultima entry coincidente tramite un consistency check, cancella le entry successive nel follower e invia le proprie. Il leader mantiene un `nextIndex` per ogni follower per gestire questo riallineamento.

9.5.4 Restrizioni e Safety

Per garantire la safety, RAFT impone una restrizione all'elezione: un votante nega il voto a un candidato se il log del candidato è meno aggiornato del proprio. L'aggiornamento si confronta controllando prima il numero del term e poi l'indice dell'ultima entry. Inoltre, un leader non effettua mai il commit di una entry di un term precedente contando solo le repliche; il commit avviene solo per le entry del term corrente, trascinando con sé per log matching le entry precedenti.

9.5.5 Disponibilità e Timing

Mentre la safety è indipendente dal tempo, la disponibilità dipende dal rispetto della relazione: $broadcastTime \ll electionTimeout \ll MTBF$ (Mean Time Between Failures). Se i messaggi sono troppo lenti rispetto ai crash, il sistema non riuscirà a eleggere un leader stabile e non farà progressi.

10 Architettura e gestione delle transazioni in SimpleDB

L'architettura interna del DBMS didattico SimpleDB è strutturata in una serie di moduli cooperanti responsabili della gestione dell'intero ciclo di vita di un'interazione con il database, dalla ricezione della query SQL alla garanzia delle proprietà transazionali. Il modulo **JDBC** funge da interfaccia verso il client, ricevendo le richieste e inoltrando il codice SQL al **Planner**. Quest'ultimo coordina l'esecuzione chiamando il **Parser** per l'analisi sintattica della stringa, determinando il piano di accesso ottimale e passandolo al modulo **Query**. Il modulo Query esegue effettivamente il piano interfacciandosi con il modulo **Record** per la manipolazione fisica dei dati.

La gestione strutturale e transazionale è affidata a componenti specializzati:

- **Metadata:** Amministra gli schemi delle tabelle e le informazioni sui cataloghi.
- **Record:** Gestisce l'organizzazione dei blocchi fisici per ospitare i record delle tabelle.
- **Concurrency Manager:** Regola l'accesso simultaneo alle risorse per evitare interferenze tra transazioni.
- **Buffer:** Mantiene le pagine di dati in memoria centrale per ottimizzare le prestazioni riducendo gli accessi al disco.
- **Recovery Manager:** Implementa i protocolli di affidabilità per garantire che i dati rimangano consistenti in caso di guasti.
- **File:** Rappresenta il livello più basso che gestisce le operazioni di lettura e scrittura fisica delle pagine sul supporto di memorizzazione.

10.1 Struttura delle classi per la gestione transazionale

L'organizzazione del sistema transazionale è rappresentata da un diagramma delle classi che vede la classe **Transaction** (nel package `simpledb.tx`) come fulcro centrale del sistema. La classe **Transaction** mantiene riferimenti diretti ai principali manager: il **ConcurrencyMgr** per il controllo degli accessi, il **RecoveryMgr** per l'affidabilità e una **BufferList** privata (`mybuffers`) che tiene traccia dei buffer attualmente fissati dalla transazione. Tra i campi statici e privati di **Transaction** troviamo `nextTxNum` per l'assegnazione sequenziale degli identificativi transazionali, `END_OF_FILE` come costante di segnalazione, e riferimenti a **FileMgr** (`fm`), **LogMgr** (`lm`) e al numero identificativo della transazione (`txnum`).

I metodi esposti da **Transaction** permettono di gestire il ciclo di vita transazionale (`commit`, `rollback`, `recover`) e le operazioni sui dati (`pin`, `unpin`, `getInt`, `getString`, `setInt`, `setString`). Esistono inoltre metodi di utilità come `size`, `append`, `blockSize`, `availableBufs` e lo statico `nextTxNumber`.

La classe **ConcurrencyMgr** (nel package `simpledb.tx.concurrency`) gestisce i lock attraverso una **LockTable** condivisa e una mappa locale `locks` che associa ogni **BlockId** al tipo di lock posseduto (rappresentato come `String`). Espone i metodi `sLock` per acquisire lock condivisi, `xLock` per lock esclusivi, `release` per liberare tutte le risorse e `hasXLock` per verificare il possesso di un lock di scrittura.

La classe **RecoveryMgr** (nel package `simpledb.tx.recovery`) coordina il logging e il ripristino. Riceve nel costruttore i riferimenti alla **Transaction** corrente, al `txnum`, al **LogMgr** e al **BufferMgr**. I suoi metodi principali includono `commit`, `rollback` e `recover`, oltre a helper per l'undo e il redo come `setInt`, `setString`, `doRollback` e `doRecover`.

La **BufferList** gestisce i buffer associati alla transazione attraverso una mappa `buffers` (tra **BlockId** e **Buffer**) e una lista `pins` di **BlockId**. Fornisce l'astrazione necessaria per il pinning e l'unpinning dei blocchi durante l'esecuzione delle operazioni.

10.2 La gestione dei Lock tramite la LockTable

La **LockTable** è il componente interno critico per la gestione dei blocchi in memoria. Essa definisce una costante `MAX_TIME` che rappresenta il tempo massimo di attesa per l'acquisizione di un lock prima di generare un timeout. La struttura dati interna principale è una **Map** che associa ogni **BlockId** a un **Integer**. Il valore intero codifica lo stato del lock:

- Un valore pari a -1 indica la presenza di un lock esclusivo (*X*).
- Un valore positivo $n > 0$ indica il numero di transazioni che possiedono un lock condiviso (*S*) su quel blocco.
- L'assenza della chiave o un valore pari a 0 indica che il blocco è libero.

I metodi della `LockTable` includono le primitive per l'acquisizione e il rilascio (`sLock`, `xLock`, `unlock`), oltre a predicati per verificare lo stato del blocco come `hasXlock` (controlla se c'è un lock esclusivo), `hasOtherSlocks` (controlla se ci sono altri lock condivisi oltre a quello della transazione corrente) e `getLockVal`. Il metodo `waitingTooLong` verifica se l'attesa corrente ha superato il limite di `MAX_TIME`.

10.3 Flussi operativi: Scansione e Modifica dei dati

Il funzionamento del sistema può essere analizzato attraverso i diagrammi di sequenza che descrivono le operazioni di lettura e scrittura.

10.3.1 Operazione di lettura (`getInt`)

Quando un client richiede la lettura di un intero tramite `Transaction.getInt(BlockId, int)`, il flusso è il seguente:

1. La `Transaction` chiama `ConcurrencyMgr.sLock(BlockId)`.
2. Il `ConcurrencyMgr` controlla la sua mappa locale `locks`. Se non possiede già un lock sul blocco, chiama `LockTable.sLock(BlockId)`. All'interno di una sezione critica, la `LockTable` aggiorna il valore del lock incrementandolo.
3. Una volta ottenuto il lock, la `Transaction` interroga la `BufferList` tramite `getBuffer(BlockId)` per ottenere l'istanza del `Buffer`.
4. La `Transaction` accede al contenuto del buffer tramite `contents()` ottenendo la `Page`.
5. Infine, viene invocato il metodo `getInt(int)` sulla `Page` (gestita da `simplifiedb.file`) per restituire il valore al chiamante.

10.3.2 Operazione di scrittura (`setInt`)

L'operazione `Transaction.setInt(BlockId, int, int, boolean)` è più complessa dovendo gestire l'affidabilità:

1. La `Transaction` chiama `ConcurrencyMgr.xLock(BlockId)`.
2. Il `ConcurrencyMgr` verifica se possiede già un lock esclusivo. In caso contrario, deve prima assicurarsi di avere un lock condiviso (eventualmente chiamando `sLock`) e poi chiamare `LockTable.xLock(BlockId)` per effettuare l'upgrade. La sezione critica della `LockTable` attende che non vi siano altri lock condivisi prima di impostare il valore a -1.
3. Dopo l'acquisizione del lock, la `Transaction` ottiene il `Buffer` dalla `BufferList`.
4. Se l'operazione deve essere registrata nel log (`okToLog`), viene chiamato `RecoveryMgr.setInt(Buffer, int, int)`.
5. Il `RecoveryMgr` accede alla `Page`, legge il valore corrente (`before state`) e chiama lo statico `SetIntRecord.writeToLog(LogMgr, int, BlockId, int, int)` per rendere persistente l'operazione di undo.
6. Infine, la `Transaction` aggiorna la `Page` tramite `setInt` e marca il buffer come modificato tramite `setModified`.

10.4 Conclusione della transazione e rilascio dei Lock

Al momento del `commit()`, la transazione deve consolidare le modifiche e liberare tutte le risorse impegnate:

1. La `Transaction` chiama `RecoveryMgr.commit()`, che assicura la scrittura dei record di log necessari e l'eventuale flush dei buffer modificati.
2. Viene invocato `ConcurrencyMgr.release()`. Questo modulo itera sulla sua collezione locale di lock.
3. Per ogni blocco, chiama `LockTable.unlock(BlockId)`. La `LockTable` decrementa il contatore dei lock condivisi o lo azzerava in caso di lock esclusivo; se il valore risultante è 0, la voce viene rimossa dalla mappa.
4. Infine, la `Transaction` ordina alla `BufferList` di eseguire `unpinAll()`, rilasciando tutti i buffer fissati durante l'esecuzione.

10.5 Strategie alternative di acquisizione dei Lock

In SimpleDB, l'acquisizione dei lock sui blocchi avviene tipicamente quando vengono invocati i metodi di accesso ai dati (`getInt` o `getString`). Una strategia alternativa proposta prevede l'acquisizione del lock condiviso nel momento esatto in cui viene effettuata la **pin** di un blocco.

L'assunzione alla base di questa strategia è che l'operazione di **pin** venga eseguita solo quando vi è l'intenzione effettiva di leggere il contenuto del blocco. I vantaggi di questo approccio includono una rilevazione più precoce dei conflitti di concorrenza, potenzialmente riducendo il tempo speso in operazioni che verranno poi bloccate. Tuttavia, lo svantaggio principale risiede in una possibile riduzione del grado di parallelismo: transazioni che necessitano di fissare molti blocchi per scopi gestionali o di scansione rapida potrebbero bloccare inutilmente risorse prima ancora di averne bisogno per il calcolo effettivo, aumentando la probabilità di stalli (deadlock) e attese superflue.